Green University of Bangladesh
Department of Computer Science and Engineering (CSE)
Faculty of Sciences and Engineering
Semester: (Spring, Year:2026), B.Sc. in CSE (Day)

LAB REPORT NO: 1
Course Title: Operating System Lab
Course Code: CSE 402 **Section:** 232-D1

Lab Experiment Name: Basic Linux Commands in Ubuntu Terminal

Student Details

Name		ID
1.	Rukonuzzaman Topu	232002280

Submission Date :
Course Teacher's Name :

[For Teachers use only: Don't Write Anything inside this box]

Lab Report Status
Marks: …… **Signature:.....**
Comments:..... **Date:.....**

1 Objective(s)

To gather knowledge of basic Linux/Unix commands in the Ubuntu terminal: file manipulation, wildcards, permissions, filters, regular expressions, piping and redirection, and process management.

2 Linux Commands

A command line (terminal) is a text based interface to the operating system. Commands are typed into a shell, most commonly Bash (Bourne Again Shell), which executes them and prints the result. This report lists 200 commonly used Linux/Unix commands, grouped by purpose. Each entry has the command name, a description, its syntax, an example and the expected output.

2.1 Terminal and Basic Navigation

These commands help you find out where you are, move around the file system and get basic information about your session.

Command 1: `pwd`

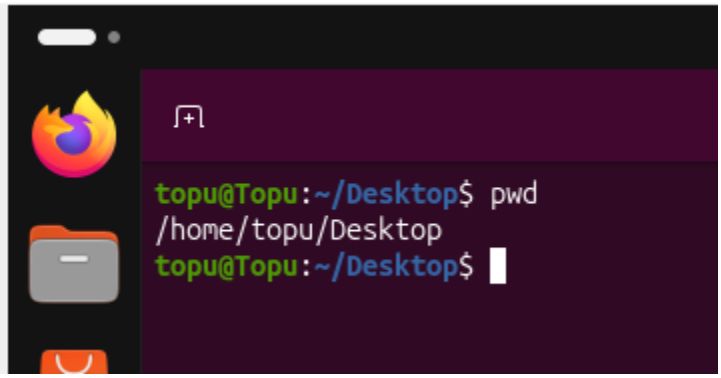
The **`pwd`** command in Linux is used to print the absolute path of the current working directory.

Syntax:

```
pwd
```

Example:

```
Pwd
```



Output: Prints the full path of the directory you are currently in.

Command 2: `ls`

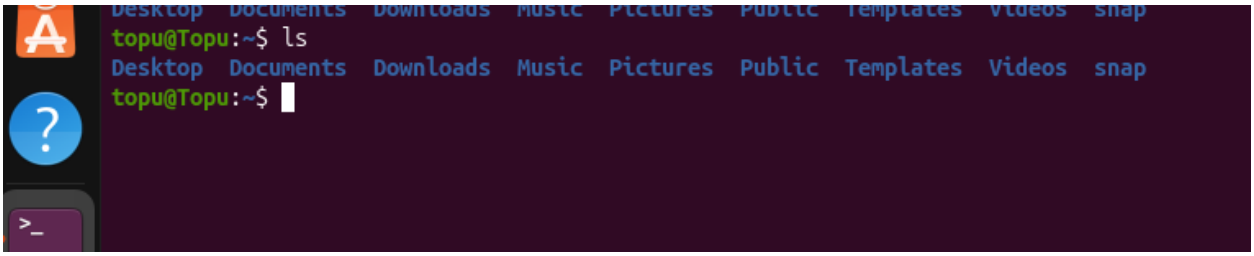
The **`ls`** command in Linux is used to list the files and directories inside a directory.

Syntax:

```
ls
```

Example:

```
ls
```



Output: Lists the contents of the Topu directory in long format.

Command 3: `cd`

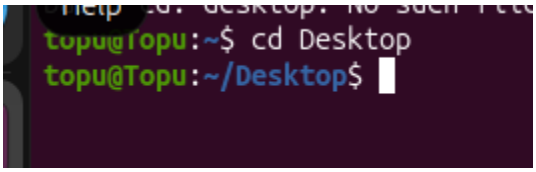
The **`cd`** command in Linux is used to change the current working directory.

Syntax:

```
cd [directory]
```

Example:

```
cd Desktop
```



Output: The prompt now shows Documents as the working directory.

Command 4: `cd ..`

The **`cd ..`** command in Linux is used to move one level up to the parent directory.

Syntax:

```
cd ..
```

Example:

```
cd ..
```



Output: The working directory changes to the parent directory.

Command 5: `cd ~`

The **`cd ~`** command in Linux is used to jump straight to the home directory.

Syntax:

```
cd ~
```

Example:

```
cd ~
```

```
topu@Topu:~/Desktop$ cd ~
topu@Topu:~$
```

Output: The working directory changes to your home directory.

Command 6: `cd -`

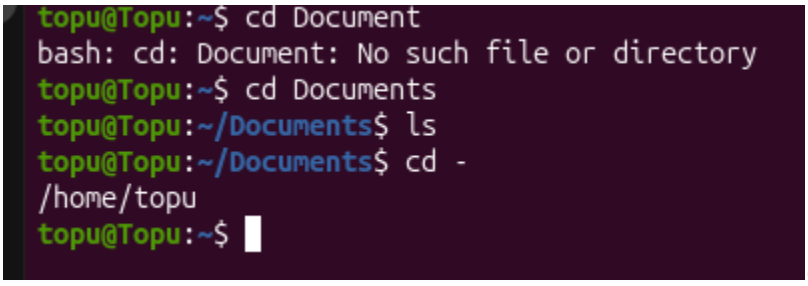
The **`cd -`** command in Linux is used to switch back to the previous working directory.

Syntax:

```
cd -
```

Example:

```
cd -
```



cle

Output: Returns to the directory you were in before and prints its path.

Command 7: `clear`

The **`clear`** command in Linux is used to clear the terminal screen.

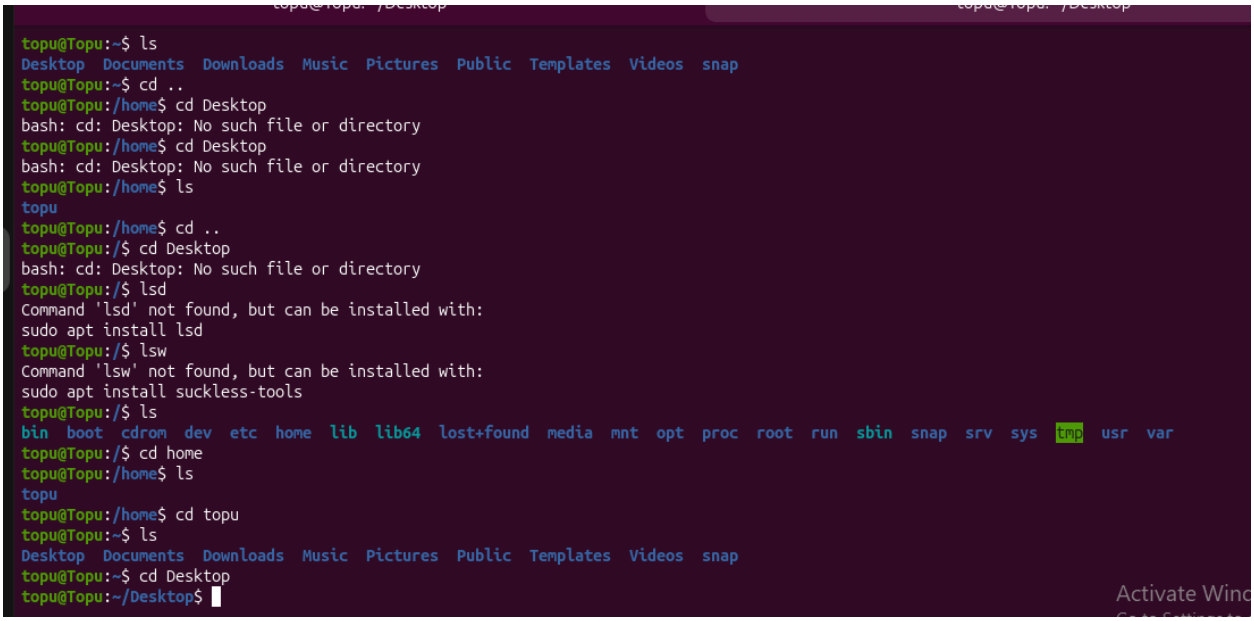
Syntax:

```
clear
```

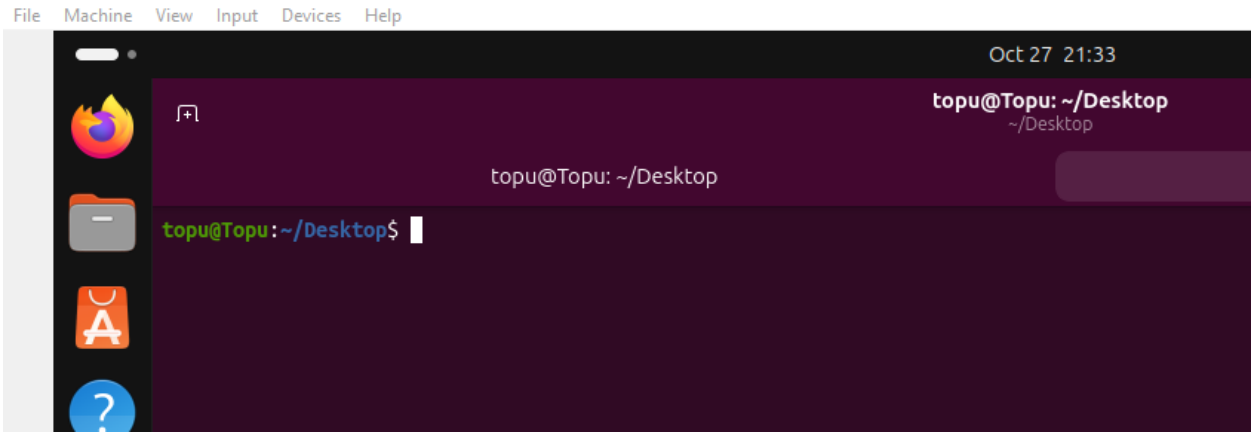
Example:

```
Clear
```

`Before`



`After`



Output: The terminal screen is cleared and the prompt moves to the top.

Command 8: `exit`

The **`exit`** command in Linux is used to close the current shell session or terminal.

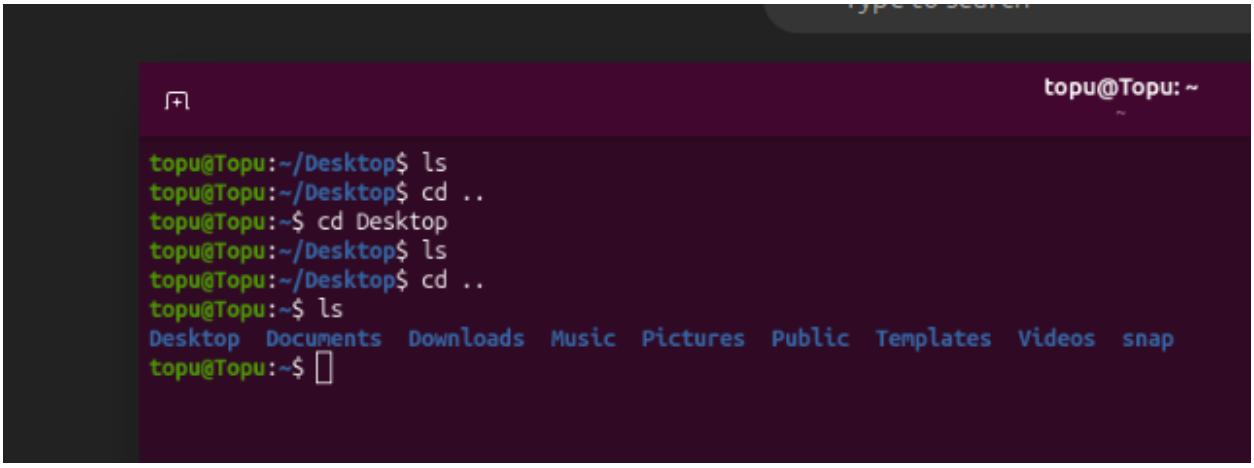
Syntax:

```
exit
```

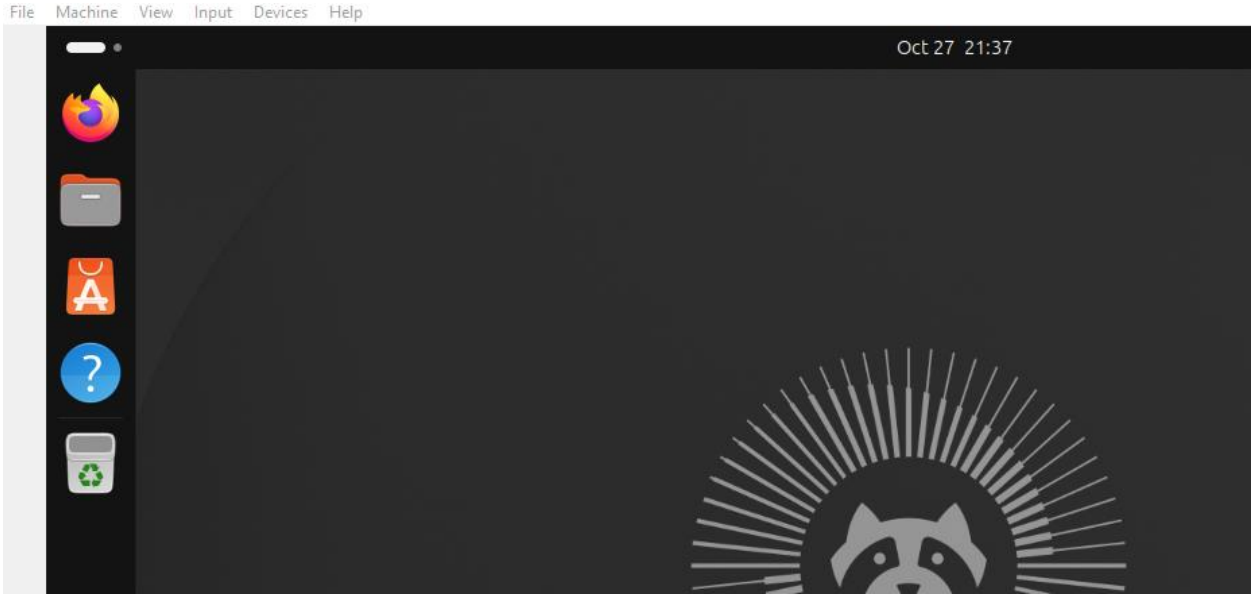
Example:

```
Exit
```

```
Before
```



`After`



Output: The terminal window or shell session is closed.

Command 9: `whoami`

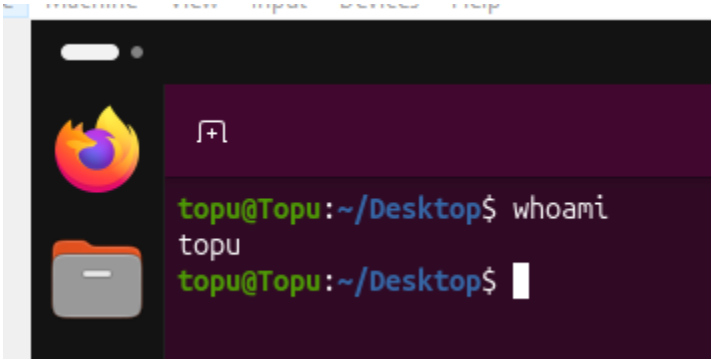
The **`whoami`** command in Linux is used to display the username of the current user.

Syntax:

```
whoami
```

Example:

```
Whoami
```



Output: The username of the logged-in user is displayed.

Command 10: `hostname`

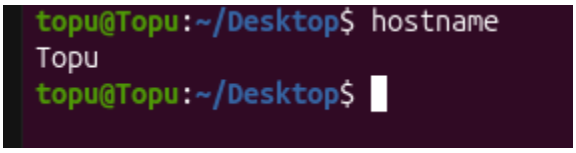
The **`hostname`** command in Linux is used to show (or set) the host name of the system.

Syntax:

```
hostname
```

Example:

```
Hostname
```



Output: The host name of the computer is displayed.

Command 11: `echo`

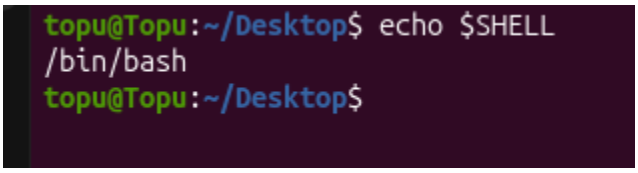
The **`echo`** command in Linux is used to print text or the value of a variable on the screen.

Syntax:

```
echo [string]
```

Example:

```
echo $SHELL
```



Output: Prints the current shell, for example /bin/bash.

Command 12: `date`

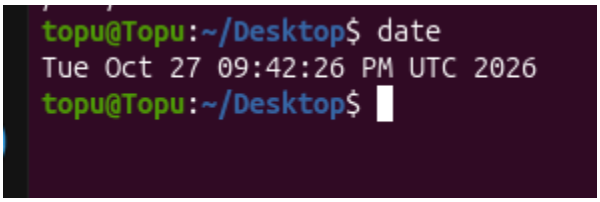
The **`date`** command in Linux is used to display or set the system date and time.

Syntax:

```
date
```

Example:

```
Date
```



Output: Prints today's date in day-month-year format.

Command 13: `cal`

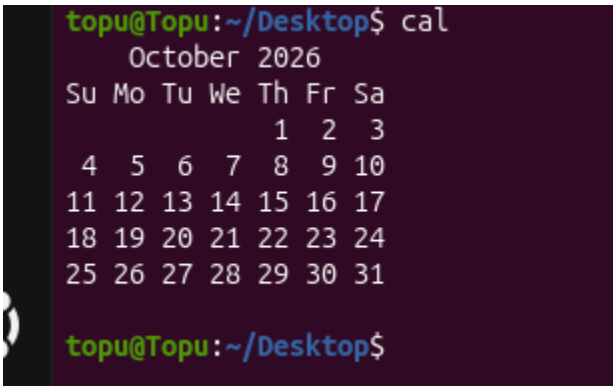
The **`cal`** command in Linux is used to display a calendar of the current or a given month.

Syntax:

```
cal [month] [year]
```

Example:

```
cal 10 2026
```



Output: Prints the calendar of October 2026.

Command 14: `history`

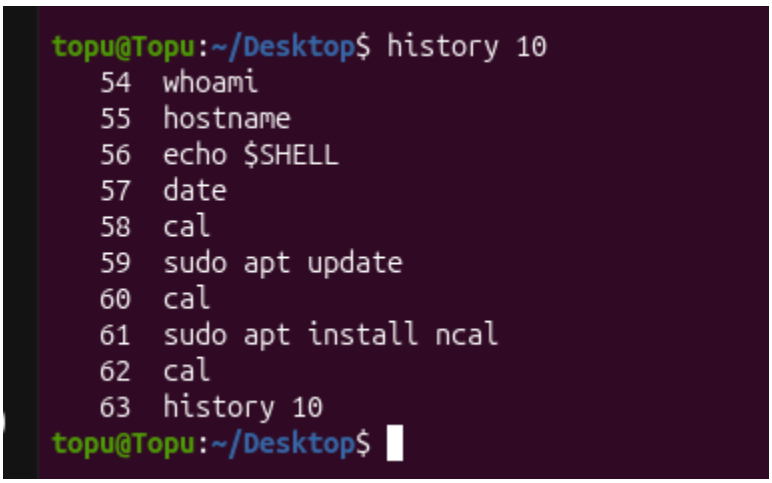
The **`history`** command in Linux is used to list the commands previously typed in the shell.

Syntax:

```
history [n]
```

Example:

```
history 10
```



Output: Shows the last 10 commands typed in the shell.

Command 15: `man`

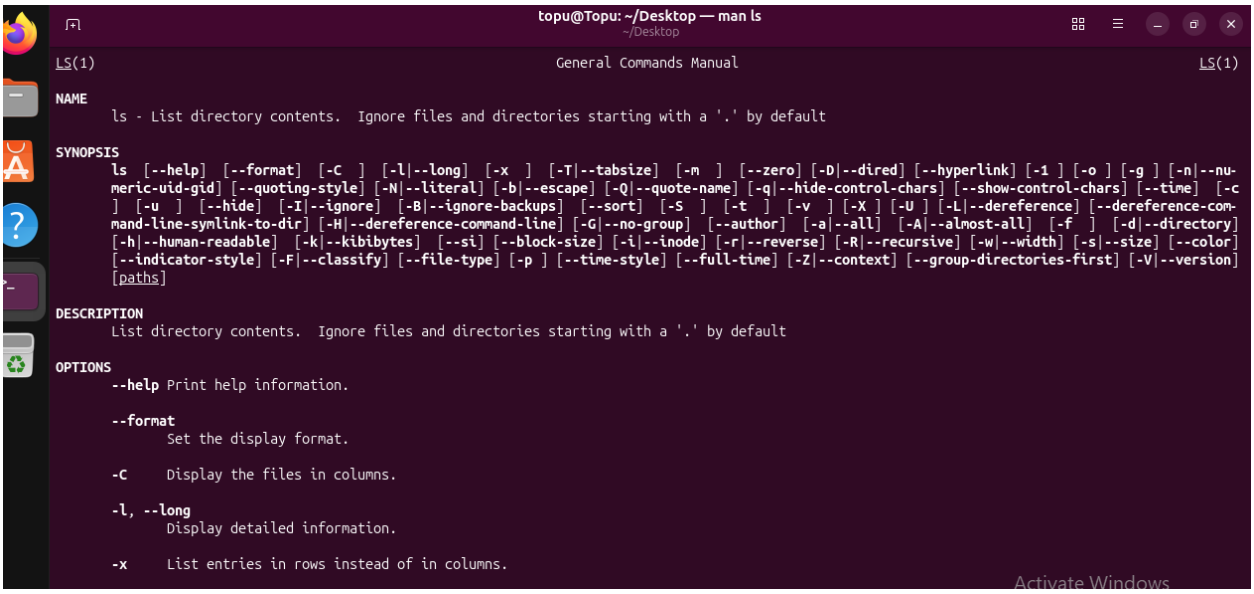
The **`man`** command in Linux is used to open the manual page of a command.

Syntax:

```
man [command]
```

Example:

```
man ls
```



Output: The manual page of the ls command opens; press q to quit.

2.2 File and Directory Manipulation

These commands are used to create, copy, move, rename, link and delete files and directories.

Command 16: `touch`

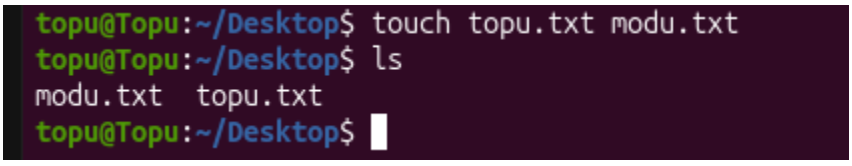
The **`touch`** command in Linux is used to create an empty file or update the timestamp of an existing file.

Syntax:

```
touch [file]
```

Example:

```
touch topu.txt modu.txt
```



Output: Creates the empty files topu.txt and modu.txt; confirm with ls.

Command 17: `mkdir`

The **`mkdir`** command in Linux is used to create a new directory.

Syntax:

```
mkdir [directory]
```

Example:

```
mkdir Topu
```

```
topu@Topu:~/Desktop$ mkdir Topu
topu@Topu:~/Desktop$ ls
Topu  modu.txt  topu.txt
topu@Topu:~/Desktop$
```

Output: A new directory named Topu is created in the current directory.

Command 18: `mkdir -p`

The **`mkdir -p`** command in Linux is used to create nested directories (and missing parents) in one step.

Syntax:

```
mkdir -p [path]
```

Example:

```
mkdir -p lab/week1/day1
```

```
topu@Topu:~/Desktop$ mkdir -p lab/week1/day1
topu@Topu:~/Desktop$ ls
Topu  lab  modu.txt  topu.txt
topu@Topu:~/Desktop$
```

Output: The directories lab, week1 and day1 are created inside each other.

Command 19: `rmdir`

The **`rmdir`** command in Linux is used to remove an empty directory.

Syntax:

```
rmdir [directory]
```

Example:

```
rmdir Green
```

```
topu@Topu:~/Desktop$ ls
Green  Topu  lab  modu.txt  topu.txt
topu@Topu:~/Desktop$ rmdir Green
topu@Topu:~/Desktop$ ls
Topu  lab  modu.txt  topu.txt
topu@Topu:~/Desktop$
```

Output: The empty directory Green is removed.

Command 20: `cp`

The **`cp`** command in Linux is used to copy a file from source to destination.

Syntax:

```
cp [source] [destination]
```

Example:

```
cp topu.txt backup.txt
```

```
topu@Topu:~/Desktop$ cp topu.txt backup.txt
topu@Topu:~/Desktop$ ls
Topu  backup.txt  lab  modu.txt  topu.txt
topu@Topu:~/Desktop$
```

Output: A copy of topu.txt named backup.txt is created.

Command 21: `cp -r`

The **`cp -r`** command in Linux is used to copy a directory and all of its contents recursively.

Syntax:

```
cp -r [source_dir] [dest_dir]
```

Example:

```
cp -r lab Example_bak
```

```
topu@Topu:~/Desktop$ cp -r lab Exampl_bak
topu@Topu:~/Desktop$
```

Output: The lab directory and all its files are copied to Example_bak.

Command 22: `mv`

The **`mv`** command in Linux is used to move or rename a file or directory.

Syntax:

```
mv [source] [destination]
```

Example:

```
mv modu.txt butterfly.txt
```

```
topu@Topu:~/Desktop$ mv modu.txt butterfly.txt
topu@Topu:~/Desktop$ ls
Exampl_bak  Topu  backup.txt  butterfly.txt  lab  topu.txt
topu@Topu:~/Desktop$
```

Output: The file modu.txt is renamed to butterfly.txt.

Command 23: `rm`

The **`rm`** command in Linux is used to delete a file.

Syntax:

```
rm [file]
```

Example:

```
rm butterfly.txt
```

```
topu@Topu:~/Desktop$ ls
Exampl_bak  Topu  backup.txt  butterfly.txt  lab  topu.txt
topu@Topu:~/Desktop$ rm butterfly.txt
topu@Topu:~/Desktop$ ls
Exampl_bak  Topu  backup.txt  lab  topu.txt
topu@Topu:~/Desktop$
```

Output: The file butterfly.txt is deleted permanently.

Command 24: `rm -r`

The **`rm -r`** command in Linux is used to delete a directory together with everything inside it.

Syntax:

```
rm -r [directory]
```

Example:

```
rm -r Exampl_bak
```

```
topu@Topu:~/Desktop$ ls
Exampl_bak  Topu  backup.txt  lab  topu.txt
topu@Topu:~/Desktop$ rm -r Exampl_bak
topu@Topu:~/Desktop$ ls
Topu  backup.txt  lab  topu.txt
topu@Topu:~/Desktop$
```

Output: The Exampl_bak directory and everything inside it are deleted.

Command 25: `rm -i`

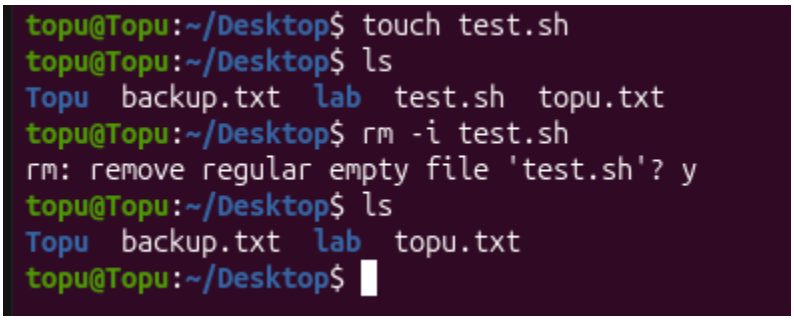
The **`rm -i`** command in Linux is used to delete files but ask for confirmation before each removal.

Syntax:

```
rm -i [file]
```

Example:

```
rm -i test.sh
```



Output: Asks for confirmation before deleting test.sh; type y to confirm.

Command 26: `ln`

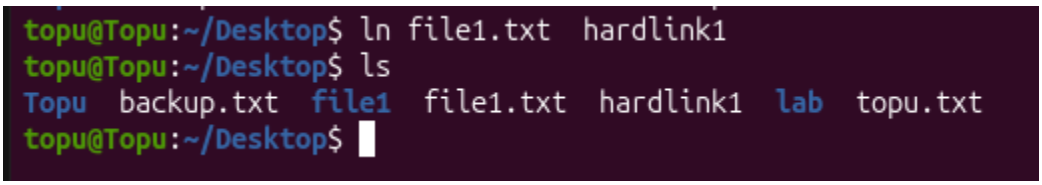
The **`ln`** command in Linux is used to create a hard link to a file.

Syntax:

```
ln [target] [link_name]
```

Example:

```
ln file1.txt hardlink1
```



Output: hardlink1 becomes another name for the same data as file1.

Command 27: `ln -s`

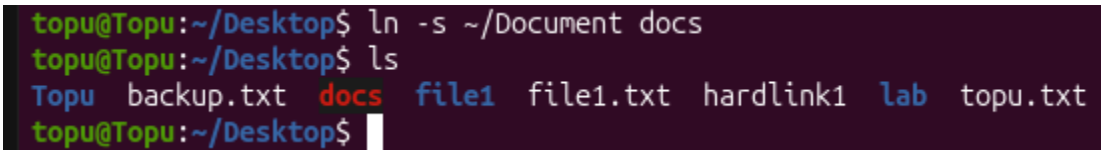
The **`ln -s`** command in Linux is used to create a symbolic (soft) link to a file or directory.

Syntax:

```
ln -s [target] [link_name]
```

Example:

```
ln -s ~/Documents docs
```



Output: A shortcut named docs that points to the Documents directory is created.

Command 28: `stat`

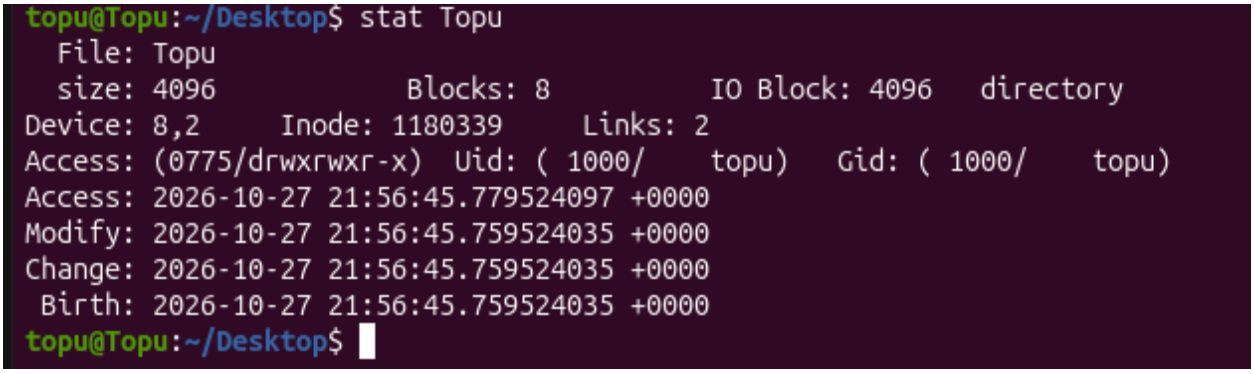
The **`stat`** command in Linux is used to display the detailed status of a file: size, inode, permissions and timestamps.

Syntax:

```
stat [file]
```

Example:

```
Stat Topu
```



Output: Shows size, permissions, inode, and access/modify times of Topu.

Command 29: `file`

The **`file`** command in Linux is used to identify the type of a file by examining its content.

Syntax:

```
file [file]
```

Example:

```
file flower.png
```

```
topu@Topu:~/Desktop$ file flower.png
flower.png: empty
topu@Topu:~/Desktop$
```

Output: Reports the file type, for example PNG image data.

Command 30: `tree`

The **`tree`** command in Linux is used to display the directory structure as a tree (install with apt if missing).

Syntax:

```
tree [directory]
```

Example:

```
tree lab
```

```
topu@Topu:~/Desktop$ tree lab
lab
└── week1
    └── day1

3 directories, 0 files
topu@Topu:~/Desktop$
```

Output: Prints all files and sub-directories of lab as a tree.

Command 31: `basename`

The **`basename`** command in Linux is used to strip the directory part from a path and print only the file name.

Syntax:

```
basename [path]
```

Example:

```
basename /home/user/report.txt
```

```
topu@Topu:~/Desktop$ basename /home/user/report.txt
report.txt
topu@Topu:~/Desktop$
```

Output: Prints only report.txt, with the directory path removed.

2.3 Viewing File Content

These commands display the content of a file in different ways.

Command 32: `catnano`

The **`cat`** command in Linux is used to display the content of a file or join several files together.

Syntax:

```
cat [file]
```

Example:

```
cat filter.txt
```

```
topu@Topu:~/Desktop$ cat filter.txt
  ct marks
name marks
topu 10
modu 9
ashab 10
rafi 8
nahid 7
mawa 10
topu@Topu:~/Desktop$
```

Output: The full content of filter.txt is printed on the screen.

Command 33: `tac`

The **`tac`** command in Linux is used to display the content of a file with the lines in reverse order.

Syntax:

```
tac [file]A
```

Example:

```
tac filter.txt
```

```
topu@Topu:~/Desktop$ tac filter.txt
mawa 10
nahid 7
rafi 8
ashab 10
modu 9
topu 10
name marks
  ct marks
topu@Topu:~/Desktop$
```

Output: The lines of filter.txt are printed from last to first.

Command 34: `more`

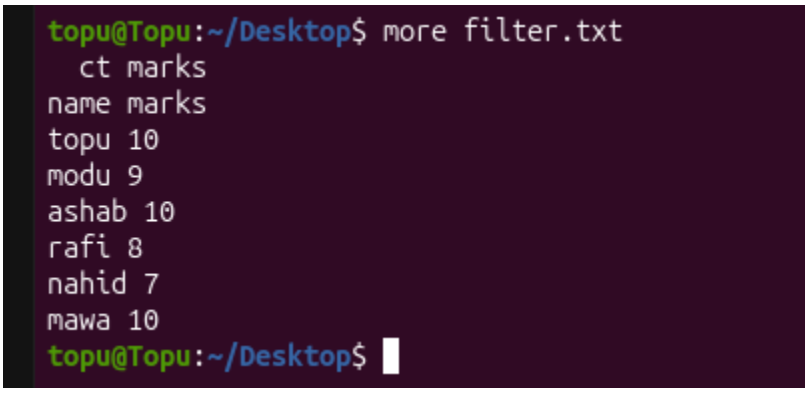
The **`more`** command in Linux is used to show a file one screen at a time (forward scrolling only).

Syntax:

```
more [file]
```

Example:

```
more filter.txt
```



Output: filter.txt is shown one screen at a time; press the space bar for the next page.

Command 35: `less`

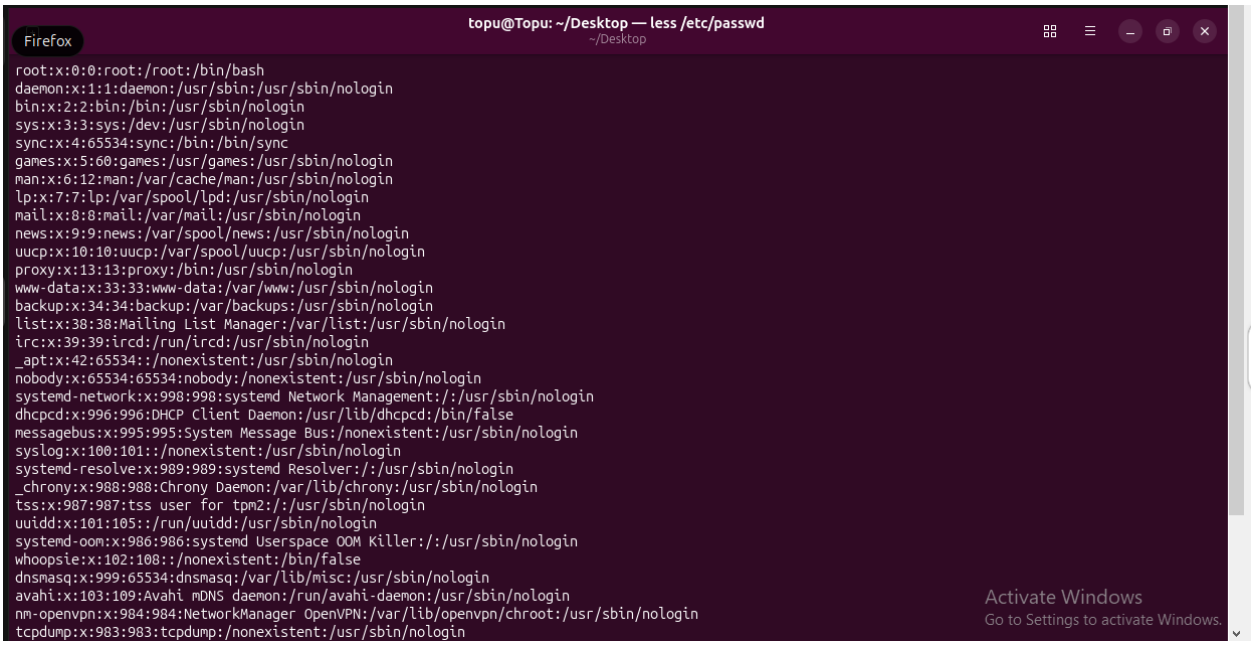
The **`less`** command in Linux is used to show a file page by page with both forward and backward scrolling.

Syntax:

```
less [file]
```

Example:

```
less /etc/passwd
```



Output: The file opens in a scrollable viewer; press q to quit.

Command 36: `head`

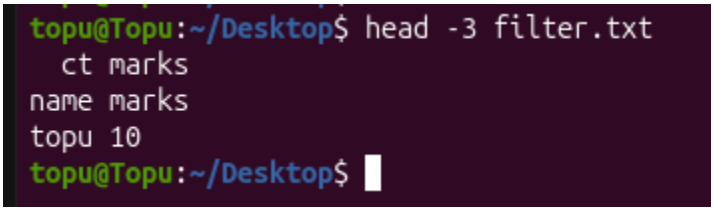
The **`head`** command in Linux is used to print the first lines of a file (10 by default).

Syntax:

```
head [-n] [file]
```

Example:

```
head -3 filter.txt
```



Output: Only the first 3 lines of filter.txt are displayed.

Command 37: `tail`

The **`tail`** command in Linux is used to print the last lines of a file (10 by default).

Syntax:

```
tail [-n] [file]
```

Example:

```
tail -5 filter.txt
```



Output: Only the last 5 lines of filter.txt are displayed.

Command 38: `tail -f`

The **`tail -f`** command in Linux is used to follow a file and print new lines as they are appended to it.

Syntax:

```
tail -f [file]
```

Example:

```
tail -f filter.txt /var/log/syslog
```



Output: The log is displayed and keeps updating as new lines arrive; press Ctrl+C to stop.

Command 39: `nl`

The **`nl`** command in Linux is used to number the lines of a file.

Syntax:

```
nl [options] [file]
```

Example:

```
nl -s '. ' -w 10 filter.txt
```

```
topu@Topu:~/Desktop$ nl -s '. ' -w 10 filter.txt
         1.   ct marks
         2. name marks
         3. topu 10
         4. modu 9
         5. ashab 10
         6. rafi 8
         7. nahid 7
         8. mawa 10
         9. trina 9
        10. diya 10
        11. tonna 9
        12. moddabir 6
        13. nothing 5
        14. abc 2
        15. efg 4
        16. xyz 0
        17. xy ab

topu@Topu:~/Desktop$
```

Output: Lines are numbered with ". " after the number and a number width of 10.

Command 40: `wc`

The **`wc`** command in Linux is used to count the lines, words and bytes of a file.

Syntax:

```
wc [options] [file]
```

Example:

```
wc -l filter.txt
```

```
topu@Topu:~/Desktop$ wc -l filter.txt
18 filter.txt
topu@Topu:~/Desktop$
```

Output: Prints the number of lines in filter.txt.

Command 41: `xxd`

The **`xxd`** command in Linux is used to show a hexadecimal dump of a file.

Syntax:

```
xxd [file]
```

Example:

```
xxd filter.txt
```



Output: Prints the bytes of filter.txt in hexadecimal next to the text.

2.4 Filters and Text Processing

A filter is a program that accepts textual data and transforms it in a particular way. These commands are often combined using pipes.

Command 42: `sort`

The **`sort`** command in Linux is used to sort the lines of text alphabetically.

Syntax:

```
sort [options] [file]
```

Example:

```
sort filter.txt
```

```
topu@Topu:~/Desktop$ sort filter.txt

abc 2
ashab 10
  ct marks
diya 10
efg 4
mawa 10
moddabir 6
modu 9
nahid 7
name marks
nothing 5
rafi 8
tonna 9
topu 10
trina 9
xy ab
xyz 0
```

Output: The lines of filter.txt are displayed in alphabetical order.

Command 43: `sort -n`

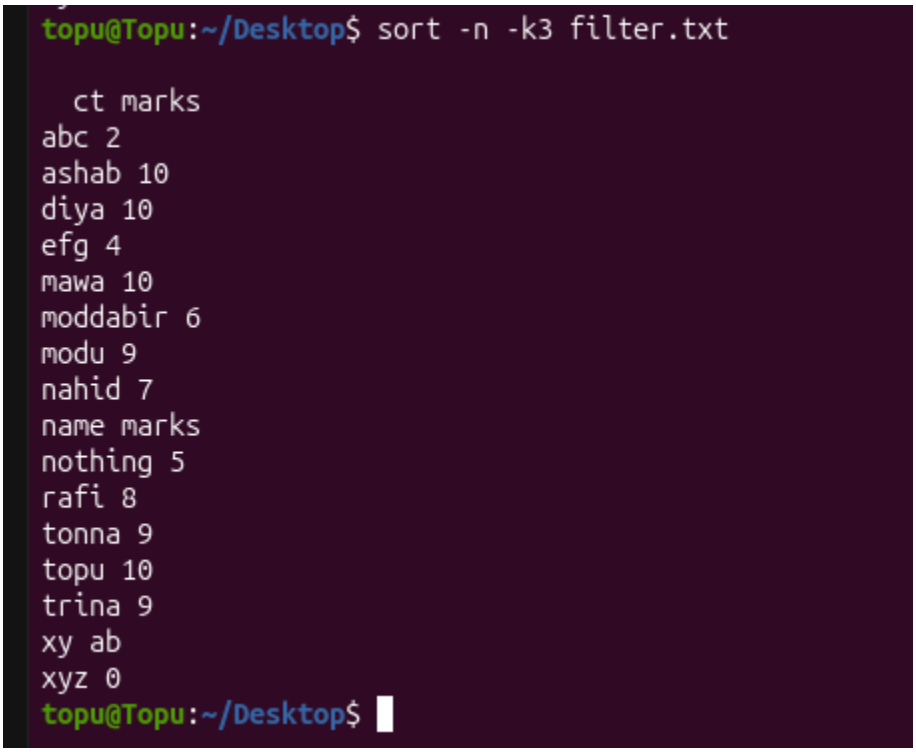
The **`sort -n`** command in Linux is used to sort the lines numerically instead of alphabetically.

Syntax:

```
sort -n [file]
```

Example:

```
sort -n -k3 filter.txt
```



Output: The lines are ordered by the number in the third column.

Command 44: `sort -r`

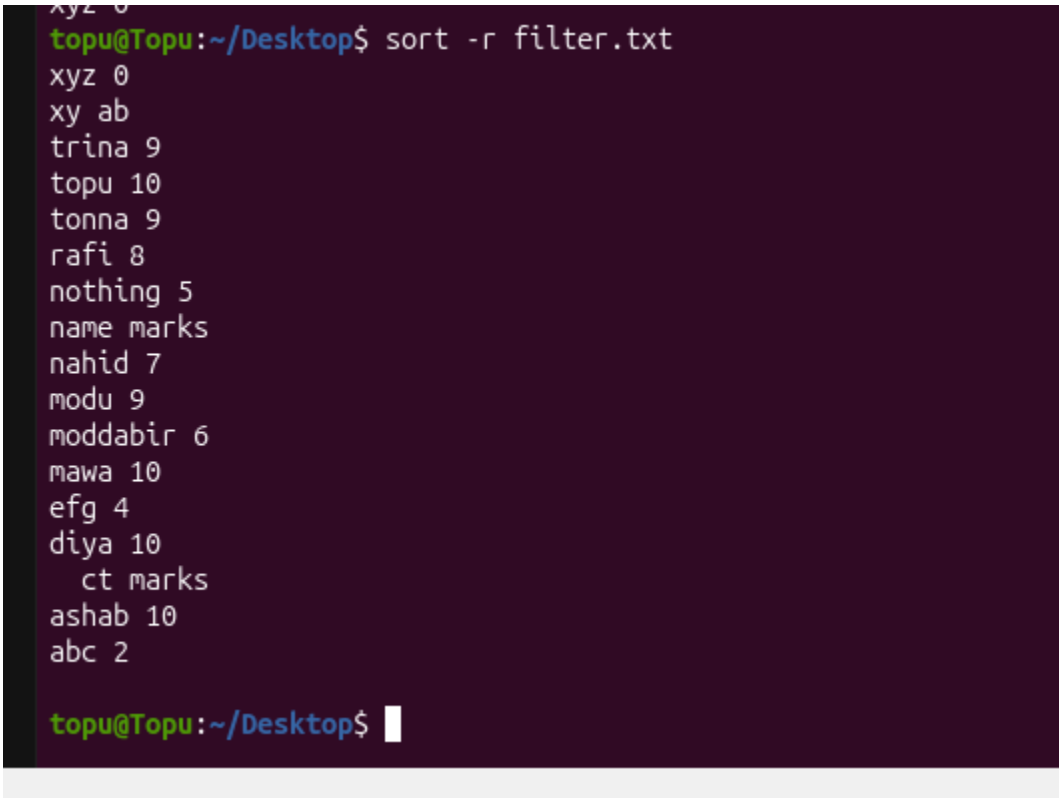
The **`sort -r`** command in Linux is used to sort the lines in reverse order.

Syntax:

```
sort -r [file]
```

Example:

```
sort -r filter.txt
```



Output: The lines are displayed in reverse alphabetical order.

Command 45: `uniq`

The **`uniq`** command in Linux is used to remove adjacent duplicate lines (input should be sorted first).

Syntax:

```
uniq [file]
```

Example:

```
sort filter.txt | uniq
```

```
topu@Topu:~/Desktop$ nano filter.txt
topu@Topu:~/Desktop$ cat filter.txt
  ct marks
name marks
topu 10
modu 9
ashab 10
rafi 8
nahid 7
mawa 10
trina 9
topu 10
topu@Topu:~/Desktop$ sort filter.txt | uniq
ashab 10
  ct marks
mawa 10
modu 9
nahid 7
name marks
rafi 8
topu 10
trina 9
topu@Topu:~/Desktop$
```

Output: Duplicate lines are removed after sorting.

Command 46: `uniq -c`

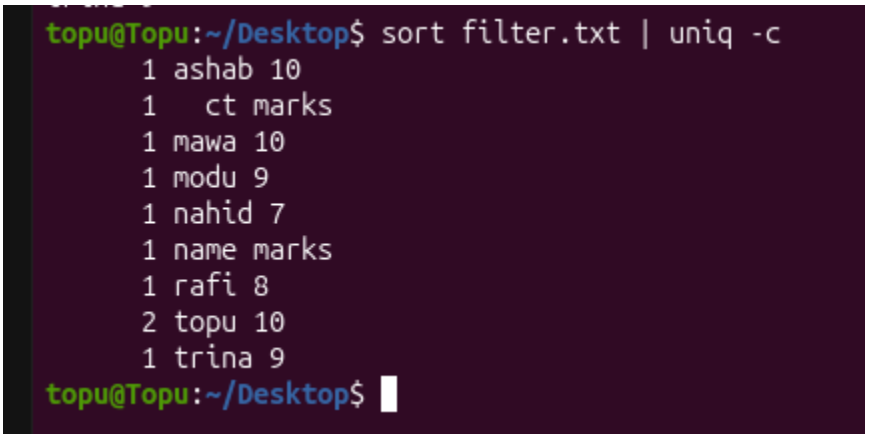
The **`uniq -c`** command in Linux is used to count how many times each line occurs.

Syntax:

```
uniq -c [file]
```

Example:

```
sort filter.txt | uniq -c
```



Output: Each unique line is printed with the number of times it occurs.

Command 47: `cut`

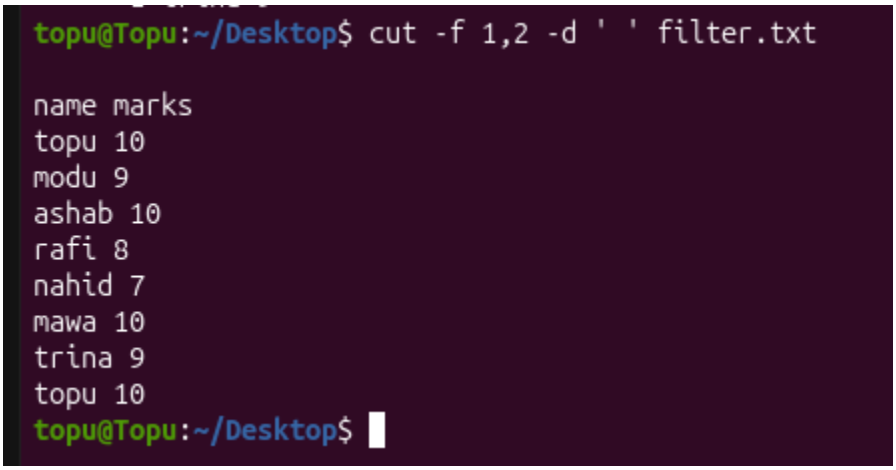
The **`cut`** command in Linux is used to extract selected fields (columns) from each line.

Syntax:

```
cut -d'[delimiter]' -f[fields] [file]
```

Example:

```
cut -f 1,2 -d ' ' filter.txt
```



Output: Only the first two columns (name and fruit) of each line are shown.

Command 48: `paste`

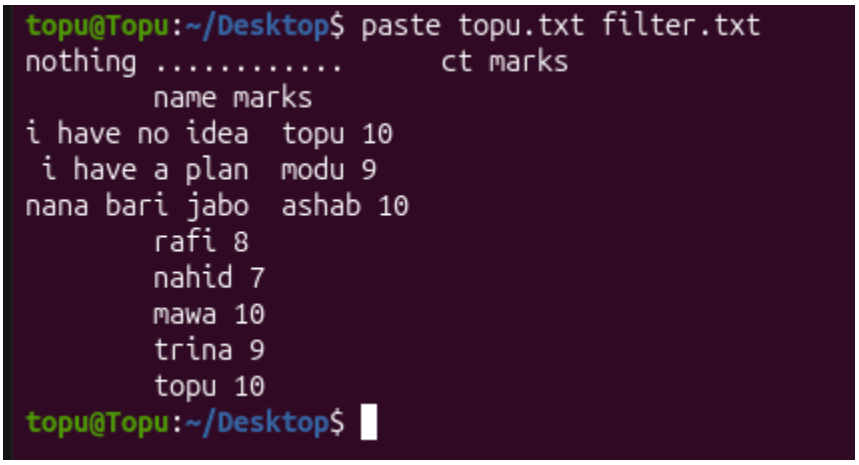
The **`paste`** command in Linux is used to merge the lines of files side by side.

Syntax:

```
paste [file1] [file2]
```

Example:

```
paste topu.txt filter.txt
```



Output: The lines of both files are printed side by side, separated by a tab.

Command 49: `join`

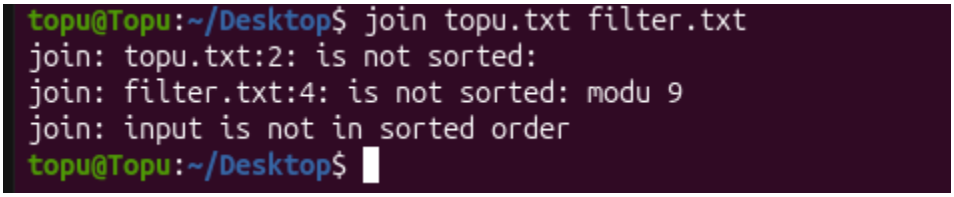
The **`join`** command in Linux is used to join the lines of two sorted files on a common field.

Syntax:

```
join [file1] [file2]
```

Example:

```
join topu.txt filter.txt
```



Output: Lines that share the same first field are combined into one line.

Command 50: `tr`

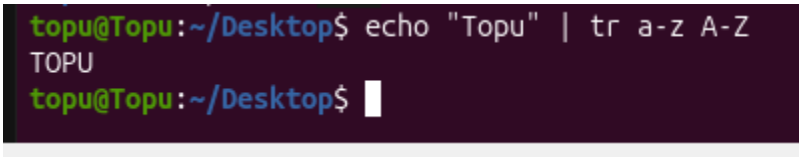
The **`tr`** command in Linux is used to translate or delete characters from the input.

Syntax:

```
tr [set1] [set2]
```

Example:

```
echo "Topu" | tr a-z A-Z
```



Output: Prints TOPU; all lowercase letters are converted to uppercase.

Command 51: `sed`

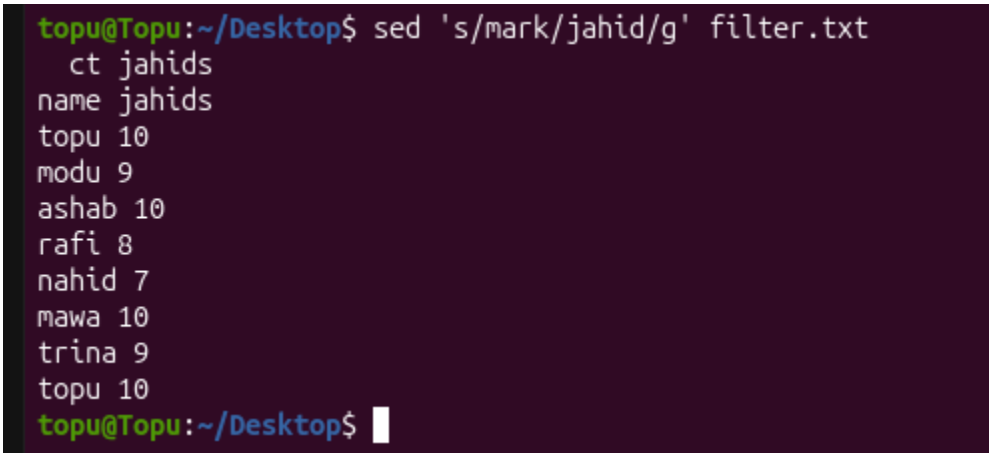
The **`sed`** command in Linux is used to search and replace text in a stream or file.

Syntax:

```
sed 's/search/replace/g' [file]
```

Example:

```
sed 's/Mark/Jahid/g' filter.txt
```



Output: Every Mark in filter.txt is shown as Jahid; the file itself is not changed.

Command 52: `awk`

The **`awk`** command in Linux is used to scan patterns and process text field by field.

Syntax:

```
awk '{print $n}' [file]
```

Example:

```
awk '{print $1, $3}' filter.txt
```



Output: Only the first and third columns (name and number) are printed.

Command 53: `tee`

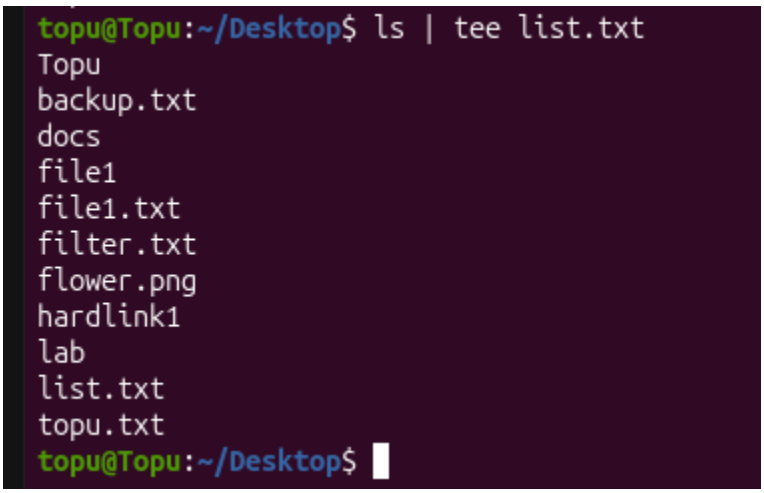
The **`tee`** command in Linux is used to read input and write it to both the screen and a file.

Syntax:

```
command | tee [file]
```

Example:

```
ls | tee list.txt
```



Output: The listing is shown on the screen and also saved in list.txt.

Command 54: `diff`

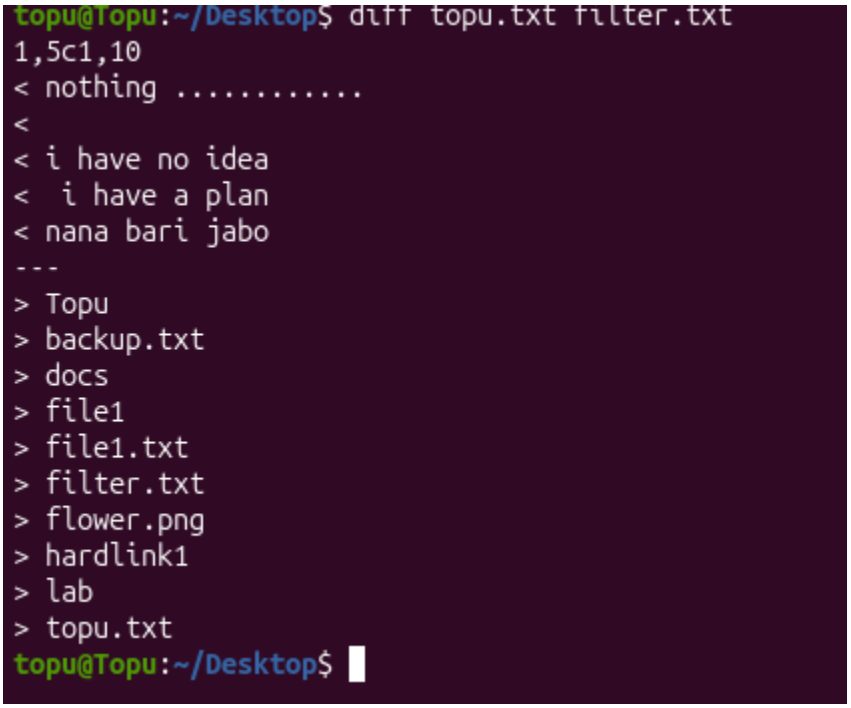
The **`diff`** command in Linux is used to compare two files line by line and show the differences.

Syntax:

```
diff [file1] [file2]
```

Example:

```
diff topu.txt filter.txt
```



Output: Lines that differ between the two files are displayed.

Command 55: `cmp`

The **`cmp`** command in Linux is used to compare two files byte by byte.

Syntax:

```
cmp [file1] [file2]
```

Example:

```
cmp topu.txt filter.txt
```



Output: Reports the first byte and line where the files differ (no output if identical).

Command 56: `comm`

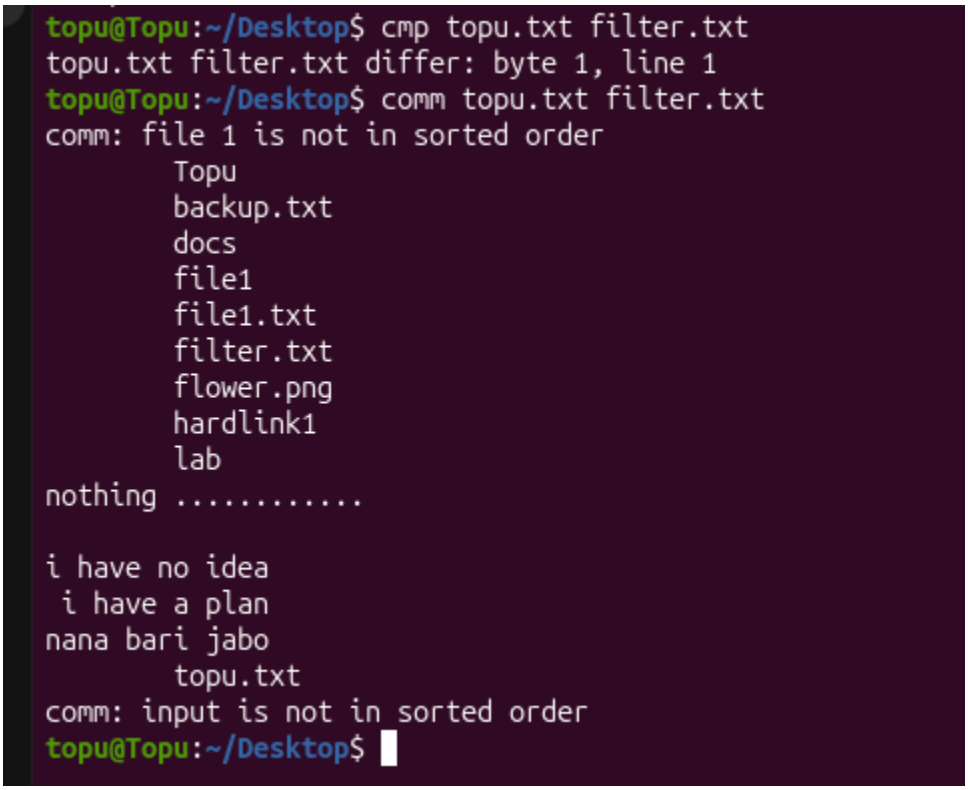
The **`comm`** command in Linux is used to compare two sorted files and print unique and common lines in columns.

Syntax:

```
comm [file1] [file2]
```

Example:

```
comm topu.txt filter.txt
```



Output: Prints three columns: lines only in file1, only in file2, and common to both.

Command 57: `rev`

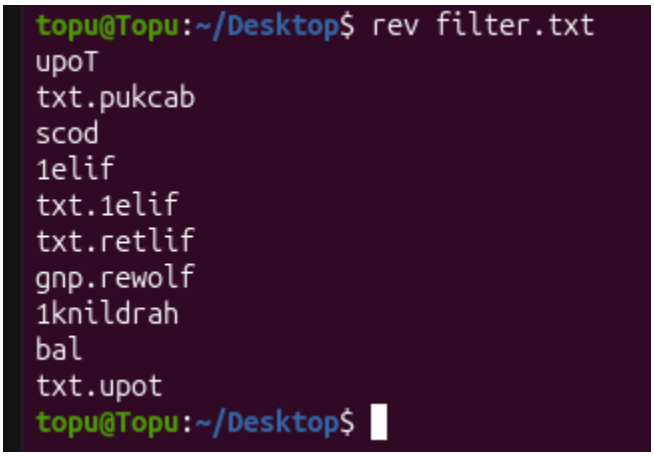
The **`rev`** command in Linux is used to reverse the characters of every line.

Syntax:

```
rev [file]
```

Example:

```
rev filter.txt
```



Output: Each line is printed with its characters in reverse order.

2.5 Searching and Regular Expressions

These commands search for text inside files and for files inside directories. Regular expressions describe the patterns to look for.

Command 58: `grep`

The **`grep`** command in Linux is used to search a file and print every line that contains a pattern.

Syntax:

```
grep [pattern] [file]
```

Example:

```
grep "mellons" RE.txt
```

Output: Lines containing mellons are printed.

Command 59: `egrep`

The **`egrep`** command in Linux is used to search for patterns using extended regular expressions.

Syntax:

```
egrep [pattern] [file]
```

Example:

```
egrep 'or|is|go' RE.txt
```

Output: Lines containing or, is or go are printed.

Command 60: `fgrep`

The **`fgrep`** command in Linux is used to search for fixed strings only, without regular expressions (same as grep -F).

Syntax:

```
fgrep [string] [file]
```

Example:

```
fgrep "2.+" RE.txt
```

Output: Lines containing the exact text 2.+ are printed; . and + are not special here.

Command 61: `grep -i`

The **`grep -i`** command in Linux is used to search while ignoring upper and lower case.

Syntax:

```
grep -i [pattern] [file]
```

Example:

```
grep -i "topu" filter.txt
```

Output: Lines containing susy in any letter case are printed.

Command 62: `grep -n`

The **`grep -n`** command in Linux is used to print matching lines together with their line numbers.

Syntax:

```
grep -n [pattern] [file]
```

Example:

```
grep -n "mellons" RE.txt
```

Output: Matching lines are printed with their line numbers.

Command 63: `grep -c`

The **`grep -c`** command in Linux is used to print only the number of matching lines.

Syntax:

```
grep -c [pattern] [file]
```

Example:

```
grep -c "mellons" RE.txt
```

Output: Only the number of matching lines is printed.

Command 64: `grep -v`

The **`grep -v`** command in Linux is used to print the lines that do not match the pattern.

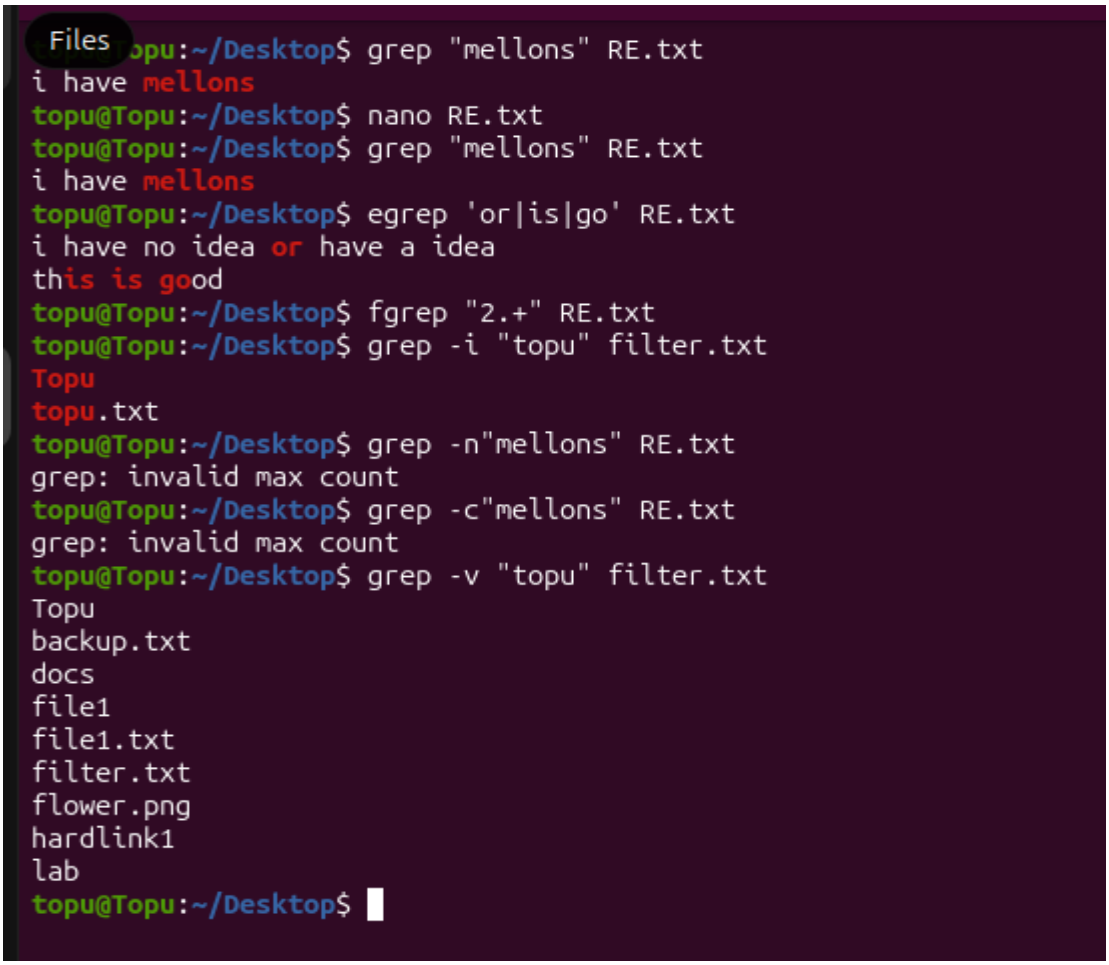
Syntax:

```
grep -v [pattern] [file]
```

Example:

```
grep -v "oranges" filter.txt
```

Output: All lines that do not contain oranges are printed.



Command 65: `grep -r`

The **`grep -r`** command in Linux is used to search recursively through all files in a directory.

Syntax:
```
grep -r [pattern] [directory]
```

Example:
```
grep -r "main" ~/lab
```

Output: Matching lines are printed together with the name of the file that contains them.

Command 66: `find`

The **`find`** command in Linux is used to search for files and directories in a directory hierarchy.

Syntax:
```
find [path] -name [pattern]
```

Example:
```
find ~ -name "*.txt"
```

Output: The paths of all .txt files under the home directory are listed.

Command 67: `locate`

The **`locate`** command in Linux is used to find files quickly using a prebuilt database.

Syntax:
```
locate [name]
```

Example:
```
locate filter.txt
```

Output: The full path of every file named filter.txt is printed.

Command 68: `which`

The **`which`** command in Linux is used to show the full path of the executable of a command.

Syntax:
```
which [command]
```

Example:
```
which python3
```

Output: Prints the path of the python3 executable, for example /usr/bin/python3.

```
topu@Topu:~/Desktop$ grep -r "main" ~/lab
grep: /home/topu/lab: No such file or directory
topu@Topu:~/Desktop$ find ~ -name "*.txt"
/home/topu/Desktop/RE.txt
/home/topu/Desktop/filter.txt
/home/topu/Desktop/list.txt
/home/topu/Desktop/backup.txt
/home/topu/Desktop/file1.txt
/home/topu/Desktop/topu.txt
topu@Topu:~/Desktop$ locate filter.txt
/home/topu/Desktop/filter.txt
/usr/lib/firmware/cxgb4/configs/t5-config-hashfilter.txt.zst
/usr/lib/firmware/cxgb4/configs/t6-config-hashfilter.txt.zst
topu@Topu:~/Desktop$ whice python3
Command 'whice' not found, did you mean:
  command 'which' from deb debianutils (5.23.2build1)
  command 'which' from deb gnu-which (2.21+dfsg-5build1)
Try: sudo apt install <deb name>
topu@Topu:~/Desktop$ which python3
/usr/bin/python3
topu@Topu:~/Desktop$
```

Command 69: `whereis`

The **`whereis`** command in Linux is used to locate the binary, source and manual page of a command.

Syntax:

```
whereis [command]
```

Example:

```
whereis ls
```

```
topu@Topu:~/Desktop$ whereis ls
ls: /usr/bin/ls /usr/share/man/man1/ls.1.gz
topu@Topu:~/Desktop$
```

Output: Prints the locations of the ls binary and its manual page.

2.6 Redirection, Piping and Wildcards

Every program has three streams: STDIN (0), STDOUT (1) and STDERR (2). The operators below connect these streams to files and to other programs. Wildcards match several file names at once.

Command 70: >

The **>** operator in Linux is used to redirect output to a file; the old content of the file is overwritten.

Syntax:
```
command > [file]
```

Example:
```
ls > myoutput
```

Output: Nothing is printed; the directory listing is saved in the file myoutput.

Command 71: >>

The **>>** operator in Linux is used to append output to the end of a file.

Syntax:
```
command >> [file]
```

Example:
```
ls >> myoutput
```

Output: The listing is added to the end of myoutput without erasing it.

Command 72: <

The **<** operator in Linux is used to take the input of a command from a file.

Syntax:
```
command < [file]
```

Example:
```
sort < filter.txt
```

Output: sort reads filter.txt as its input and prints the sorted lines.

Command 73: 2>

The **2>** operator in Linux is used to redirect error messages (STDERR) to a file.

Syntax:
```
command 2> [file]
```

Example:
```
ls nofile 2> error.log
```

Output: The error message is saved in error.log instead of being shown on the screen.

Command 74: `2>&1`

The **`2>&1`** operator in Linux is used to send STDERR to the same place as STDOUT.

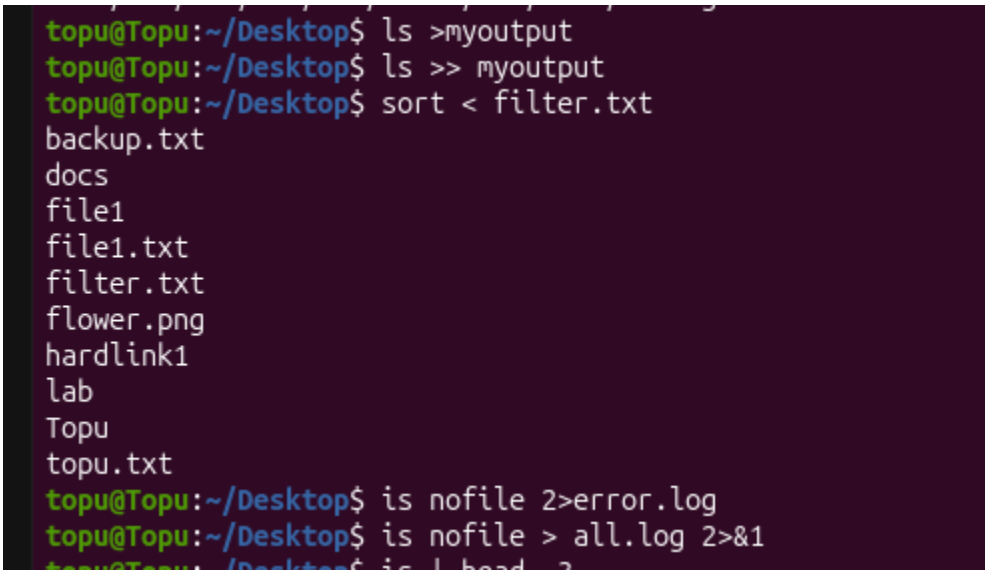
Syntax:

```
command > [file] 2>&1
```

Example:

```
ls nofile > all.log 2>&1
```

Output: Both normal output and error messages are saved in all.log.



Command 75: `|`

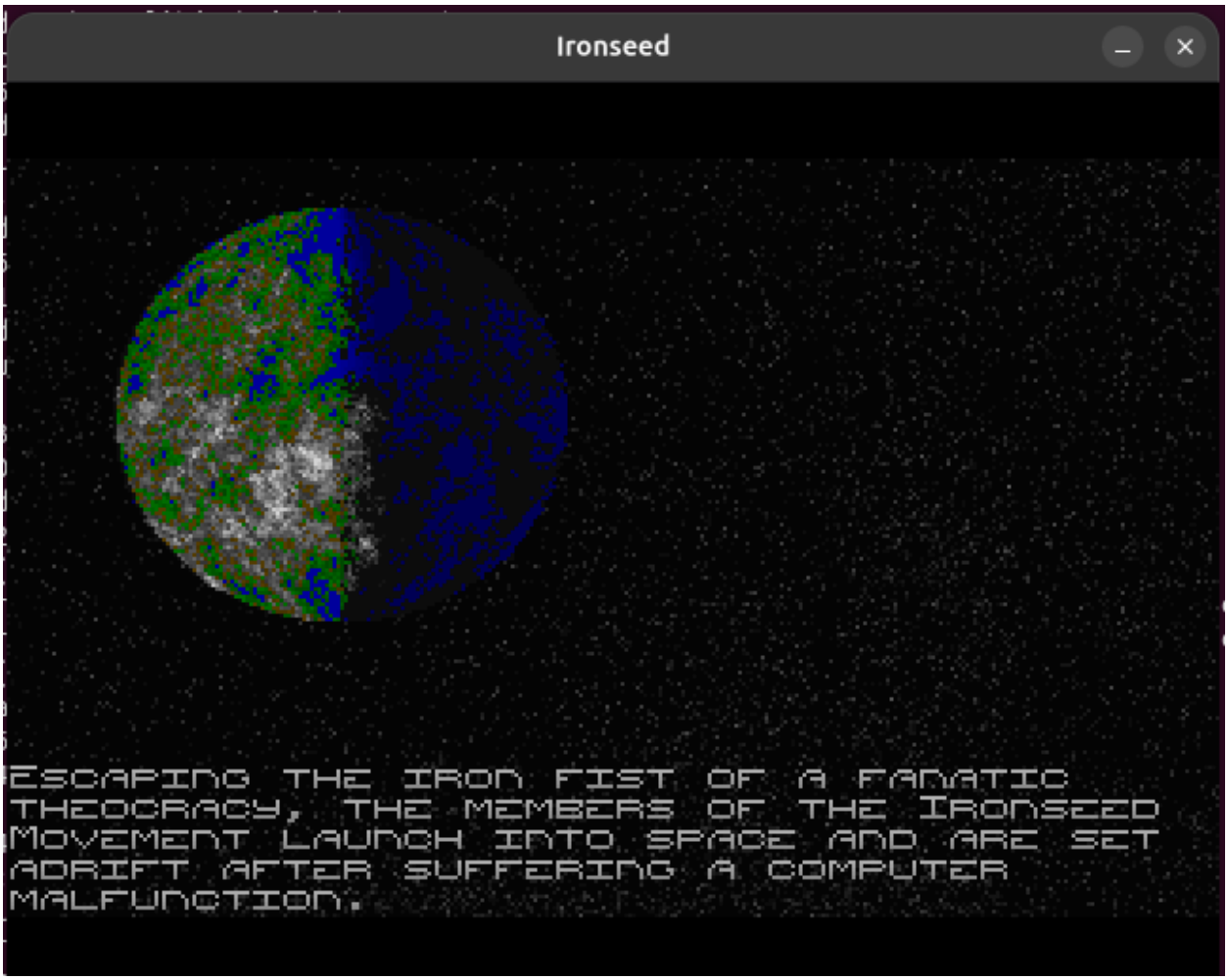
The `|` operator in Linux is used to pass the output of one command as the input of the next command.

Syntax:

```
command1 | command2
```

Example:

```
ls | head -3
```



Output: Only the first three entries of the listing are displayed.

Command 76: `&&`

The `&&` operator in Linux is used to run the second command only if the first one succeeds.

Syntax:

```
command1 && command2
```

Example:

```
mkdir test && cd test
```

Output: cd runs only if mkdir succeeded.

Command 77: `||`

The `||` operator in Linux is used to run the second command only if the first one fails.

Syntax:

```
command1 || command2
```

Example:

```
cd test || echo "not found"
```

Output: The message is printed only if the cd command fails.

Command 78: `;`

The `;` operator in Linux is used to run commands one after another, whether or not the previous one succeeds.

Syntax:

```
command1 ; command2
```

Example:

```
pwd ; ls
```

Output: Both commands run one after the other.

Command 79: `xargs`

The **`xargs`** command in Linux is used to build and run commands using items read from standard input.

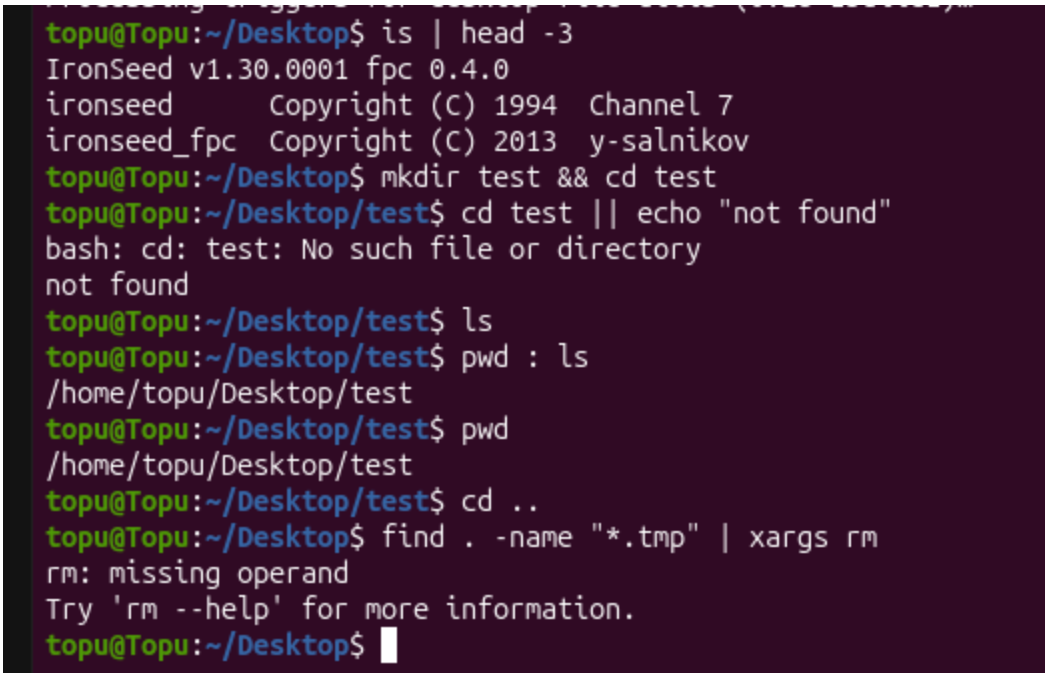
Syntax:

```
command | xargs [command]
```

Example:

```
find . -name "*.tmp" | xargs rm
```

Output: All .tmp files found in the current directory tree are deleted.



Command 80: <<

The **<<** operator in Linux is used to pass several lines of text to a command as input until a delimiter word is reached.

Syntax:

```
command << [DELIMITER]
```

Example:

```
cat << EOF
```

Output: The lines you type are read as input until you type EOF on a line by itself.

Command 81: *

The ***** wildcard in Linux is used to match zero or more characters in a file name.

Syntax:

```
ls [pattern]
```

Example:

```
ls b*
```

Output: Lists every file whose name begins with b.

Command 82: `?`

The `?` wildcard in Linux is used to match exactly one character in a file name.

Syntax:

```
ls [pattern]
```

Example:

```
ls ?i*
```

Output: Lists files whose second character is i, such as firstfile.

Command 83: `[ ]`

The `[ ]` wildcard in Linux is used to match one character from a set or a range of characters.

Syntax:

```
ls [pattern]
```

Example:

```
ls [a-v]*
```

Output: Lists files whose names start with a letter from a to v.



2.7 Permissions and Ownership

Linux permissions control what the owner, the group and others may do with a file: read (r), write (w) and execute (x).

Command 84: `ls -l`

The `ls -l` command in Linux is used to list files in long format showing permissions, owner, group, size and date.

Syntax:

```
ls -l [path]
```

Example:

```
ls -l frog.png
```

Output: Shows permissions such as -rwxr-----x along with owner, group, size and date.

Command 85: `chmod`

The **`chmod`** command in Linux is used to change file permissions using symbolic mode (who, +/-, permission).

Syntax:

```
chmod [ugoa][+-][rwx] [file]
```

Example:

```
chmod g+x flower.png
```

Output: Execute permission is added for the group; check with ls -l.

Command 86: `chmod +x`

The **`chmod +x`** command in Linux is used to make a file executable, for example a shell script.

Syntax:

```
chmod +x [file]
```

Example:

```
chmod +x addition.sh
```

Output: The script becomes executable and can be run with ./addition.sh.

Command 87: `chmod 755`

The **`chmod 755`** command in Linux is used to set permissions using numeric (octal) mode.

Syntax:

```
chmod [mode] [file]
```

Example:

```
chmod 755 addition.sh
```

Output: The owner gets rwx; the group and others get r-x.

Command 88: `chmod -R`

The **`chmod -R`** command in Linux is used to change permissions of a directory and its contents recursively.

Syntax:

```
chmod -R [mode] [directory]
```

Example:

```
chmod -R u+w Documents
```

Output: The owner gets write permission on Documents and everything inside it.

Command 89: `chown`

The **`chown`** command in Linux is used to change the owner of a file.

Syntax:
```
sudo chown [user] [file]
```

Example:
```
sudo chown root test.sh
```

Output: The owner of test.sh becomes root; ls -l shows the change.

Command 90: `chgrp`

The **`chgrp`** command in Linux is used to change the group that owns a file.

Syntax:
```
sudo chgrp [group] [file]
```

Example:
```
sudo chgrp users frog.png
```

Output: The group of frog.png becomes users.

Command 91: `umask`

The **`umask`** command in Linux is used to display or set the default permission mask for new files.

Syntax:
```
umask [mode]
```

Example:
```
umask 022
```

Output: New files will be created with default permissions 644 (directories 755).

Command 92: `sudo`

The **`sudo`** command in Linux is used to run a command with superuser (administrator) privileges.

Syntax:
```
sudo [command]
```

Example:
```
sudo apt update
```

Output: Runs the update command as administrator after asking for your password.

Command 93: `su`

The **`su`** command in Linux is used to switch to another user account.

Syntax:

```
su [username]
```

Example:

```
su - root
```

Output: Asks for the root password and opens a shell as root; type exit to return.

Command 94: `id`

The **`id`** command in Linux is used to display the user ID and group IDs of a user.

Syntax:

```
id [username]
```

Example:

```
id
```

Output: Prints your user ID, group ID and the groups you belong to.

Command 95: `groups`

The **`groups`** command in Linux is used to show the groups a user belongs to.

Syntax:

```
groups [username]
```

Example:

```
groups
```

Output: Prints the names of the groups the current user belongs to.

2.8 Process Management

Linux is a multitasking operating system. These commands show what is running, control running programs and manage background jobs.

Command 96: `ps`

The **`ps`** command in Linux is used to show the processes running in the current terminal.

Syntax:

```
ps
```

Example:

```
ps
```

Output: Shows the PID, terminal, time and name of processes in this terminal.

Command 97: `ps aux`

The **`ps aux`** command in Linux is used to show all running processes of all users in detail.

Syntax:
```
ps aux
```

Example:
```
ps aux | grep 'firefox'
```

Output: Only the lines of processes related to firefox are displayed.

Command 98: `top`

The **`top`** command in Linux is used to show a real-time view of processes and resource usage.

Syntax:
```
top
```

Example:
```
top
```

Output: A live, updating table of processes with CPU and memory use; press q to quit.

Command 99: `htop`

The **`htop`** command in Linux is used to view running processes interactively.

Syntax:
```
htop
```

Example:
```
htop
```

Output: A colourful interactive process viewer opens; press F10 to quit.

Command 100: `kill`

The **`kill`** command in Linux is used to terminate a process using its process ID (PID).

Syntax:
```
kill [PID]
```

Example:
```
kill 6310
```

Output: The process with PID 6310 is asked to terminate.

Command 101: `kill -9`

The **`kill -9`** command in Linux is used to forcefully kill a process using the SIGKILL signal.

Syntax:
```
kill -9 [PID]
```

Example:
```
kill -9 6310
```

Output: The process 6310 is stopped immediately.

Command 102: `killall`

The **`killall`** command in Linux is used to kill all processes with a given name.

Syntax:
```
killall [name]
```

Example:
```
killall firefox
```

Output: All firefox processes are terminated.

Command 103: `pkill`

The **`pkill`** command in Linux is used to kill processes whose names match a pattern.

Syntax:
```
pkill [pattern]
```

Example:
```
pkill fire
```

Output: Processes whose names match fire are terminated.

Command 104: `pgrep`

The **`pgrep`** command in Linux is used to find the process IDs of processes by name.

Syntax:
```
pgrep [name]
```

Example:
```
pgrep firefox
```

Output: Prints the PIDs of firefox processes.

Command 105: `pidof`

The **`pidof`** command in Linux is used to print the PID of a running program.

Syntax:
```
pidof [program]
```

Example:
```
pidof bash
```

Output: Prints the PID of each running bash shell.

Command 106: `jobs`

The **`jobs`** command in Linux is used to list the background and stopped jobs of the current shell.

Syntax:
```
jobs
```

Example:
```
jobs
```

Output: Lists background and stopped jobs with their job numbers.

Command 107: `bg`

The **`bg`** command in Linux is used to resume a stopped job in the background.

Syntax:
```
bg [%job]
```

Example:
```
bg %1
```

Output: Job number 1 continues running in the background.

Command 108: `fg`

The **`fg`** command in Linux is used to bring a background job to the foreground.

Syntax:
```
fg [%job]
```

Example:
```
fg %1
```

Output: Job number 1 comes to the foreground.

Command 109: `nohup`

The **`nohup`** command in Linux is used to run a command that keeps running after the terminal is closed.

Syntax:

```
nohup [command] &
```

Example:

```
nohup ./addition.sh &
```

Output: The script keeps running after the terminal is closed; output goes to nohup.out.

Command 110: `nice`

The **`nice`** command in Linux is used to start a process with a chosen scheduling priority.

Syntax:

```
nice -n [value] [command]
```

Example:

```
nice -n 10 ./addition.sh
```

Output: The script runs with a lower priority (nice value 10).

Command 111: `renice`

The **`renice`** command in Linux is used to change the priority of an already running process.

Syntax:

```
renice [value] -p [PID]
```

Example:

```
renice 5 -p 1234
```

Output: The priority of process 1234 is changed to 5.

2.9 System Information

These commands report details about the operating system, hardware, memory and the shell environment.

Command 112: `uname -a`

The **`uname -a`** command in Linux is used to print kernel name, version and machine architecture.

Syntax:

```
uname -a
```

Example:

```
uname -a
```

Output: Prints kernel name, host name, kernel version and architecture.

Command 113: `lsb_release -a`

The **`lsb_release -a`** command in Linux is used to show Ubuntu distribution and release information.

Syntax:
```
lsb_release -a
```

Example:
```
lsb_release -a
```

Output: Prints the distributor ID, description and release of Ubuntu.

Command 114: `lscpu`

The **`lscpu`** command in Linux is used to display CPU architecture information.

Syntax:
```
lscpu
```

Example:
```
lscpu
```

Output: Prints CPU model, number of cores, threads and speed.

Command 115: `free -h`

The **`free -h`** command in Linux is used to show used and free memory in human-readable units.

Syntax:
```
free -h
```

Example:
```
free -h
```

Output: Prints total, used and free RAM and swap in readable units.

Command 116: `lshw`

The **`lshw`** command in Linux is used to list detailed hardware information.

Syntax:
```
sudo lshw [options]
```

Example:
```
sudo lshw -short
```

Output: Prints a short table of the hardware components.

Command 117: `lsusb`

The **`lsusb`** command in Linux is used to list the USB devices connected to the system.

Syntax:
```
lsusb
```

Example:
```
lsusb
```

Output: Lists the connected USB devices.

Command 118: `lspci`

The **`lspci`** command in Linux is used to list the PCI devices (graphics, network cards, etc.).

Syntax:
```
lspci
```

Example:
```
lspci
```

Output: Lists PCI devices such as graphics and network cards.

Command 119: `dmesg`

The **`dmesg`** command in Linux is used to print kernel messages, useful for hardware and boot problems.

Syntax:
```
dmesg
```

Example:
```
dmesg | tail
```

Output: Prints the last 10 kernel messages.

Command 120: `env`

The **`env`** command in Linux is used to display all environment variables.

Syntax:
```
env
```

Example:
```
env
```

Output: Prints all environment variables such as HOME and PATH.

Command 121: `printenv`

The **`printenv`** command in Linux is used to print the value of a given environment variable.

Syntax:
```
printenv [VARIABLE]
```

Example:
```
printenv HOME
```

Output: Prints the value of HOME, for example /home/<username>.

Command 122: `export`

The **`export`** command in Linux is used to set an environment variable for child processes.

Syntax:
```
export [NAME=value]
```

Example:
```
export PATH=$PATH:~/bin
```

Output: ~/bin is added to PATH for this session and its child processes.

Command 123: `alias`

The **`alias`** command in Linux is used to create a shortcut name for a longer command.

Syntax:
```
alias [name]='[command]'
```

Example:
```
alias ll='ls -l'
```

Output: Typing ll now runs ls -l (valid for the current terminal session).

2.10 Disk and Storage

These commands report disk usage and manage disks and file systems.

Command 124: `df -h`

The **`df -h`** command in Linux is used to show disk space usage of mounted file systems.

Syntax:
```
df -h
```

Example:
```
df -h
```

Output: Shows the size, used and free space of each file system.

Command 125: `du -sh`

The **`du -sh`** command in Linux is used to show the total size of a directory.

Syntax:
```
du -sh [directory]
```

Example:
```
du -sh Documents
```

Output: Prints a single total size for the Documents directory.

Command 126: `du -h`

The **`du -h`** command in Linux is used to show the size of every file and sub-directory.

Syntax:
```
du -h [directory]
```

Example:
```
du -h Documents
```

Output: Prints the size of every sub-directory and file in Documents.

Command 127: `lsblk`

The **`lsblk`** command in Linux is used to list block devices such as disks and partitions.

Syntax:
```
lsblk
```

Example:
```
lsblk
```

Output: Prints a tree of disks and partitions.

Command 128: `mount`

The **`mount`** command in Linux is used to attach a file system to a directory.

Syntax:
```
sudo mount [device] [directory]
```

Example:
```
sudo mount /dev/sdb1 /mnt
```

Output: If the device exists, its files appear in /mnt.

Command 129: `umount`

The **`umount`** command in Linux is used to detach a mounted file system.

Syntax:
```
sudo umount [directory]
```

Example:
```
sudo umount /mnt
```

Output: The partition is detached from /mnt.

Command 130: `fdisk -l`

The **`fdisk -l`** command in Linux is used to list disk partition tables.

Syntax:
```
sudo fdisk -l
```

Example:
```
sudo fdisk -l
```

Output: Lists every disk with its partitions and sizes.

Command 131: `blkid`

The **`blkid`** command in Linux is used to show the UUID and file system type of block devices.

Syntax:
```
sudo blkid
```

Example:
```
sudo blkid
```

Output: Prints the UUID and file system type of each partition.

Command 132: `sync`

The **`sync`** command in Linux is used to flush file system buffers, writing cached data to disk.

Syntax:
```
sync
```

Example:
```
sync
```

Output: Nothing is printed; cached data is written to disk.

Command 133: `fsck`

The **`fsck`** command in Linux is used to check and repair a file system (run on unmounted disks).

Syntax:
```
sudo fsck [device]
```

Example:
```
sudo fsck /dev/sdb1
```

Output: The file system is checked and errors are reported (the device must be unmounted).

2.11 Networking

These commands test connectivity, inspect network settings and transfer data between computers.

Command 134: `ip addr`

The **`ip addr`** command in Linux is used to show the IP addresses of all network interfaces.

Syntax:
```
ip addr show
```

Example:
```
ip addr show
```

Output: Lists the network interfaces with their IP addresses.

Command 135: `ifconfig`

The **`ifconfig`** command in Linux is used to display or configure network interfaces (needs net-tools).

Syntax:
```
ifconfig
```

Example:
```
ifconfig
```

Output: Shows interface details such as IP address, netmask and packet counts.

Command 136: `ping`

The **`ping`** command in Linux is used to test whether a host is reachable.

Syntax:
```
ping -c [count] [host]
```

Example:
```
ping -c 4 google.com
```

Output: Sends 4 packets and shows the reply time and packet loss.

Command 137: `traceroute`

The **`traceroute`** command in Linux is used to show the route packets take to reach a host.

Syntax:
```
traceroute [host]
```

Example:
```
traceroute google.com
```

Output: Lists each router (hop) between your computer and google.com.

Command 138: `netstat`

The **`netstat`** command in Linux is used to display network connections and listening ports.

Syntax:
```
netstat -tuln
```

Example:
```
netstat -tuln
```

Output: Lists the listening TCP and UDP ports.

Command 139: `ss`

The **`ss`** command in Linux is used to display socket statistics (modern replacement for netstat).

Syntax:
```
ss -tuln
```

Example:
```
ss -tuln
```

Output: Lists the listening TCP and UDP sockets.

Command 140: `curl`

The **`curl`** command in Linux is used to transfer data from or to a URL.

Syntax:
```
curl [options] [URL]
```

Example:
```
curl -I https://example.com
```

Output: Prints only the HTTP response headers of the site.

Command 141: `wget`

The **`wget`** command in Linux is used to download files from the web.

Syntax:
```
wget [URL]
```

Example:
```
wget https://example.com/file.zip
```

Output: The file is downloaded to the current directory with a progress bar.

Command 142: `nslookup`

The **`nslookup`** command in Linux is used to query DNS to find the IP address of a domain.

Syntax:
```
nslookup [domain]
```

Example:
```
nslookup google.com
```

Output: Prints the DNS server used and the IP address of google.com.

Command 143: `dig`

The **`dig`** command in Linux is used to perform detailed DNS lookups.

Syntax:
```
dig [domain]
```

Example:
```
dig google.com
```

Output: Prints the full DNS query and answer sections.

Command 144: `host`

The **`host`** command in Linux is used to perform a simple DNS lookup.

Syntax:
```
host [domain]
```

Example:
```
host google.com
```

Output: Prints the IP addresses of google.com.

Command 145: `ssh`

The **`ssh`** command in Linux is used to log in securely to a remote computer.

Syntax:
```
ssh [user]@[host]
```

Example:
```
ssh student@192.168.1.10
```

Output: Asks for the password and opens a shell on the remote computer.

Command 146: `scp`

The **`scp`** command in Linux is used to securely copy files between computers.

Syntax:
```
scp [file] [user]@[host]:[path]
```

Example:
```
scp file.txt student@192.168.1.10:/home/student/
```

Output: The local file is copied to the remote home directory.

Command 147: `rsync`

The **`rsync`** command in Linux is used to efficiently synchronise files and directories.

Syntax:
```
rsync -av [source] [destination]
```

Example:
```
rsync -av Documents/ backup/
```

Output: Files in Documents are copied to backup with their attributes; later runs copy only changes.

2.12 Package Management

Ubuntu uses the APT package manager (and dpkg) to install and maintain software.

Command 148: `apt update`

The **`apt update`** command in Linux is used to refresh the list of available packages.

Syntax:
```
sudo apt update
```

Example:
```
sudo apt update
```

Output: The package lists are refreshed from the repositories.

Command 149: `apt upgrade`

The **`apt upgrade`** command in Linux is used to upgrade all installed packages to newer versions.

Syntax:
```
sudo apt upgrade
```

Example:
```
sudo apt upgrade
```

Output: Installed packages are upgraded after you confirm.

Command 150: `apt install`

The **`apt install`** command in Linux is used to install a package.

Syntax:
```
sudo apt install [package]
```

Example:
```
sudo apt install tree
```

Output: The tree package is downloaded and installed.

Command 151: `apt remove`

The **`apt remove`** command in Linux is used to remove a package but keep its configuration files.

Syntax:
```
sudo apt remove [package]
```

Example:
```
sudo apt remove tree
```

Output: The tree program is removed but its settings stay.

Command 152: `apt purge`

The **`apt purge`** command in Linux is used to remove a package together with its configuration files.

Syntax:
```
sudo apt purge [package]
```

Example:
```
sudo apt purge tree
```

Output: tree and its configuration files are removed.

Command 153: `apt autoremove`

The **`apt autoremove`** command in Linux is used to remove packages that are no longer needed.

Syntax:

```
sudo apt autoremove
```

Example:

```
sudo apt autoremove
```

Output: Unused dependency packages are removed.

Command 154: `apt search`

The **`apt search`** command in Linux is used to search the package lists for a keyword.

Syntax:

```
apt search [keyword]
```

Example:

```
apt search editor
```

Output: Lists packages whose name or description contains editor.

Command 155: `apt show`

The **`apt show`** command in Linux is used to show details about a package.

Syntax:

```
apt show [package]
```

Example:

```
apt show vim
```

Output: Shows the version, size, dependencies and description of vim.

Command 156: `dpkg -i`

The **`dpkg -i`** command in Linux is used to install a local .deb package file.

Syntax:

```
sudo dpkg -i [file.deb]
```

Example:

```
sudo dpkg -i package.deb
```

Output: The package is installed from the local .deb file.

Command 157: `dpkg -l`

The **`dpkg -l`** command in Linux is used to list all installed packages.

Syntax:
```
dpkg -l
```

Example:
```
dpkg -l | grep vim
```

Output: Only installed packages that contain vim in their name are listed.

2.13 User and Group Management

These commands create and manage user accounts and groups and show who is logged in.

Command 158: `useradd`

The **`useradd`** command in Linux is used to create a new user account.

Syntax:
```
sudo useradd -m [username]
```

Example:
```
sudo useradd -m student1
```

Output: A new user student1 with a home directory is created.

Command 159: `adduser`

The **`adduser`** command in Linux is used to create a new user interactively with a home directory.

Syntax:
```
sudo adduser [username]
```

Example:
```
sudo adduser student2
```

Output: Asks for a password and details, then creates the user.

Command 160: `usermod`

The **`usermod`** command in Linux is used to modify an existing user account.

Syntax:
```
sudo usermod [options] [username]
```

Example:
```
sudo usermod -aG sudo student1
```

Output: student1 is added to the sudo group.

Command 161: `userdel`

The **`userdel`** command in Linux is used to delete a user account.

Syntax:
```
sudo userdel -r [username]
```

Example:
```
sudo userdel -r student1
```

Output: The user and their home directory are removed.

Command 162: `passwd`

The **`passwd`** command in Linux is used to change a user's password.

Syntax:
```
passwd [username]
```

Example:
```
passwd
```

Output: Asks for the current and the new password.

Command 163: `groupadd`

The **`groupadd`** command in Linux is used to create a new group.

Syntax:
```
sudo groupadd [group]
```

Example:
```
sudo groupadd students
```

Output: A new group called students is created.

Command 164: `groupdel`

The **`groupdel`** command in Linux is used to delete a group.

Syntax:
```
sudo groupdel [group]
```

Example:
```
sudo groupdel students
```

Output: The group students is removed.

Command 165: `who`

The **`who`** command in Linux is used to show who is currently logged in.

Syntax:

```
who
```

Example:

```
who
```

Output: Lists logged-in users with their terminal and login time.

Command 166: `w`

The **`w`** command in Linux is used to show who is logged in and what they are doing.

Syntax:

```
w
```

Example:

```
w
```

Output: Lists logged-in users and what they are doing.

Command 167: `last`

The **`last`** command in Linux is used to show the history of user logins.

Syntax:

```
last [-n]
```

Example:

```
last -n 5
```

Output: Shows the last 5 login records.

2.14 Archive and Compression

These commands bundle several files into one archive and compress files to save space.

Command 168: `tar -cvf`

The **`tar -cvf`** command in Linux is used to create a tar archive.

Syntax:

```
tar -cvf [archive.tar] [files]
```

Example:

```
tar -cvf backup.tar Documents
```

Output: Documents is packed into backup.tar and the file names are listed.

Command 169: `tar -xvf`

The **`tar -xvf`** command in Linux is used to extract a tar archive.

Syntax:
```
tar -xvf [archive.tar]
```

Example:
```
tar -xvf backup.tar
```

Output: Files are extracted from backup.tar into the current directory.

Command 170: `tar -czvf`

The **`tar -czvf`** command in Linux is used to create a gzip-compressed tar archive.

Syntax:
```
tar -czvf [archive.tar.gz] [files]
```

Example:
```
tar -czvf backup.tar.gz Documents
```

Output: Documents is packed and compressed into backup.tar.gz.

Command 171: `tar -xzvf`

The **`tar -xzvf`** command in Linux is used to extract a .tar.gz archive.

Syntax:
```
tar -xzvf [archive.tar.gz]
```

Example:
```
tar -xzvf backup.tar.gz
```

Output: backup.tar.gz is decompressed and extracted.

Command 172: `gzip`

The **`gzip`** command in Linux is used to compress a file into a .gz file.

Syntax:
```
gzip [file]
```

Example:
```
gzip filter.txt
```

Output: filter.txt is replaced by the compressed file filter.txt.gz.

Command 173: `gunzip`

The **`gunzip`** command in Linux is used to decompress a .gz file.

Syntax:
```
gunzip [file.gz]
```

Example:
```
gunzip filter.txt.gz
```

Output: filter.txt.gz is restored to filter.txt.

Command 174: `bzip2`

The **`bzip2`** command in Linux is used to compress a file using the bzip2 algorithm.

Syntax:
```
bzip2 [file]
```

Example:
```
bzip2 filter.txt
```

Output: filter.txt is replaced by filter.txt.bz2.

Command 175: `zip`

The **`zip`** command in Linux is used to create a zip archive.

Syntax:
```
zip -r [archive.zip] [files]
```

Example:
```
zip -r docs.zip Documents
```

Output: Documents is compressed into docs.zip.

Command 176: `unzip`

The **`unzip`** command in Linux is used to extract a zip archive.

Syntax:
```
unzip [archive.zip]
```

Example:
```
unzip docs.zip
```

Output: The contents of docs.zip are extracted.

Command 177: `xz`

The **`xz`** command in Linux is used to compress a file using the xz algorithm.

Syntax:
```
xz [file]
```

Example:
```
xz filter.txt
```

Output: filter.txt is replaced by filter.txt.xz.

2.15 Editors and Shell Scripting

These commands are used to edit text files and to write and run simple Bash scripts such as addition.sh.

Command 178: `vi`

The **`vi`** command in Linux is used to open the Vi editor (press i to insert, Esc then :wq to save and exit).

Syntax:
```
vi [file]
```

Example:
```
vi Firstfile
```

Output: Opens the file in Vi; press i to type, then Esc and :wq to save and quit.

Command 179: `vim`

The **`vim`** command in Linux is used to open Vim, an improved version of Vi.

Syntax:
```
vim [file]
```

Example:
```
vim addition.sh
```

Output: Opens the script in Vim.

Command 180: `nano`

The **`nano`** command in Linux is used to open the beginner-friendly Nano editor.

Syntax:
```
nano [file]
```

Example:
```
nano notes.txt
```

Output: Opens the editor; press Ctrl+O to save and Ctrl+X to exit.

Command 181: `gedit`

The **`gedit`** command in Linux is used to open the GNOME graphical text editor.

Syntax:
```
gedit [file]
```

Example:
```
gedit addition.sh
```

Output: Opens the script in the graphical gedit window.

Command 182: `read`

The **`read`** command in Linux is used to read a line of input into a shell variable.

Syntax:
```
read [variable]
```

Example:
```
read a
```

Output: Waits for you to type a value and stores it in the variable a.

Command 183: `expr`

The **`expr`** command in Linux is used to evaluate an arithmetic or string expression.

Syntax:
```
expr [expression]
```

Example:
```
expr 10 + 20
```

Output: Prints 30.

Command 184: `bc`

The **`bc`** command in Linux is used to perform calculations, including decimals, from the command line.

Syntax:
```
echo "[expression]" | bc
```

Example:
```
echo "scale=2; 10/3" | bc
```

Output: Prints 3.33.

Command 185: `source`

The **`source`** command in Linux is used to run a script in the current shell.

Syntax:
```
source [file]
```

Example:
```
source ~/.bashrc
```

Output: Reloads the settings in .bashrc into the current shell.

Command 186: `test`

The **`test`** command in Linux is used to evaluate a condition (file checks, comparisons).

Syntax:
```
test [condition]
```

Example:
```
test -f filter.txt && echo "exists"
```

Output: Prints exists only when filter.txt is a regular file.

Command 187: `bash`

The **`bash`** command in Linux is used to run a script using the Bash shell.

Syntax:
```
bash [script]
```

Example:
```
bash addition.sh
```

Output: Runs the script without needing execute permission.

2.16 Miscellaneous Utilities

A few more useful commands that do not fit neatly into the earlier groups.

Command 188: `uptime`

The **`uptime`** command in Linux is used to show how long the system has been running and the load average.

Syntax:
```
uptime
```

Example:
```
uptime
```

Output: Prints the current time, how long the system has been up, users and load average.

Command 189: `pstree`

The **`pstree`** command in Linux is used to display running processes as a tree.

Syntax:
```
pstree
```

Example:
```
pstree
```

Output: Shows parent and child processes as a tree.

Command 190: `lsof`

The **`lsof`** command in Linux is used to list open files and the processes that use them.

Syntax:
```
sudo lsof -i :[port]
```

Example:
```
sudo lsof -i :22
```

Output: Lists the processes that use port 22.

Command 191: `watch`

The **`watch`** command in Linux is used to run a command repeatedly and show its output.

Syntax:
```
watch -n [seconds] [command]
```

Example:
```
watch -n 2 df -h
```

Output: Runs df -h every 2 seconds; press Ctrl+C to stop.

Command 192: `sleep`

The **`sleep`** command in Linux is used to pause for a given number of seconds.

Syntax:
```
sleep [seconds]
```

Example:
```
sleep 5
```

Output: The terminal waits 5 seconds before showing the prompt again.

Command 193: `time`

The **`time`** command in Linux is used to measure how long a command takes to run.

Syntax:
```
time [command]
```

Example:
```
time ls
```

Output: Prints the listing, then the real, user and sys times.

Command 194: `seq`

The **`seq`** command in Linux is used to print a sequence of numbers.

Syntax:
```
seq [first] [last]
```

Example:
```
seq 1 5
```

Output: Prints the numbers 1 to 5, one per line.

Command 195: `printf`

The **`printf`** command in Linux is used to print formatted text.

Syntax:
```
printf "[format]" [args]
```

Example:
```
printf "%s\n" "Hello"
```

Output: Prints Hello followed by a new line.

Command 196: `unalias`

The **`unalias`** command in Linux is used to remove an alias.

Syntax:
```
unalias [name]
```

Example:
```
unalias ll
```

Output: The ll shortcut no longer works.

Command 197: `shuf`

The **`shuf`** command in Linux is used to shuffle the lines of input randomly.

Syntax:
```
shuf [file]
```

Example:
```
shuf filter.txt
```

Output: The lines of filter.txt are printed in random order.

Command 198: `vmstat`

The **`vmstat`** command in Linux is used to report virtual memory, CPU and I/O statistics.

Syntax:
```
vmstat [delay] [count]
```

Example:
```
vmstat 2 5
```

Output: Prints memory, CPU and I/O statistics 5 times at 2-second intervals.

Command 199: `timedatectl`

The **`timedatectl`** command in Linux is used to show or change the system time and date settings.

Syntax:
```
timedatectl
```

Example:
```
timedatectl
```

Output: Prints local time, UTC time and time zone settings.

Command 200: `yes`

The **`yes`** command in Linux is used to repeatedly print a string until it is stopped.

Syntax:
```
yes [string]
```

Example:
```
yes "ok" | head -3
```

Output: Prints ok three times.

3 Discussion & Conclusion

In this lab I practised 200 Linux/Unix commands covering navigation, file and directory handling, text filters, regular expressions, redirection and piping, permissions and process management. Combining small commands with pipes and redirection showed me how powerful the command

line is for processing data and automating work. Based on the focused objective(s) to understand the basic Linux commands, this exercise made me more confident towards the fulfilment of the objective(s).